



Chương 3

Thiết kế bộ nhớ

System-on-Chip Design

Mục lục

3.1. TỔNG QUAN

3.2. BỘ NHỚ TRÊN CHIP

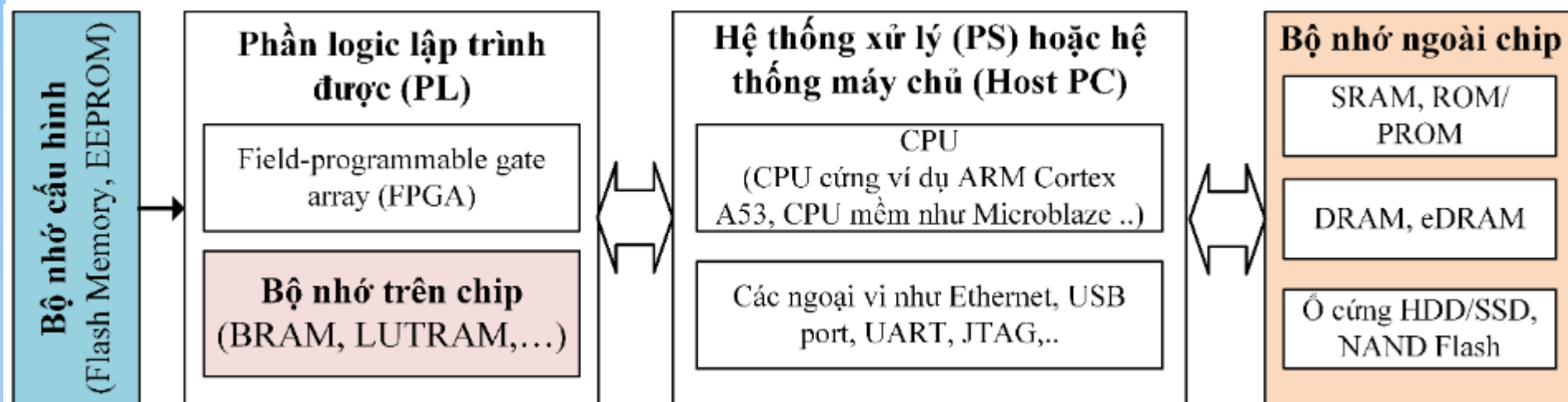
- 3.2.1. Giới thiệu
- 3.2.2. Block RAM
- 3.2.3. Ultra RAM
- 3.2.4. LUTRAM hay Distributed RAM
- 3.2.5. Bộ đệm cache
- 3.2.6. FIFO

3.3. BỘ NHỚ NGOÀI CHIP

- 3.3.1. Giới thiệu
- 3.3.2. Bộ nhớ DRAM
- 3.3.3. Bộ nhớ NAND flash
- 3.3.4. Bộ nhớ cấu hình (Configuration Memory)

3.4. Câu hỏi và Bài tập

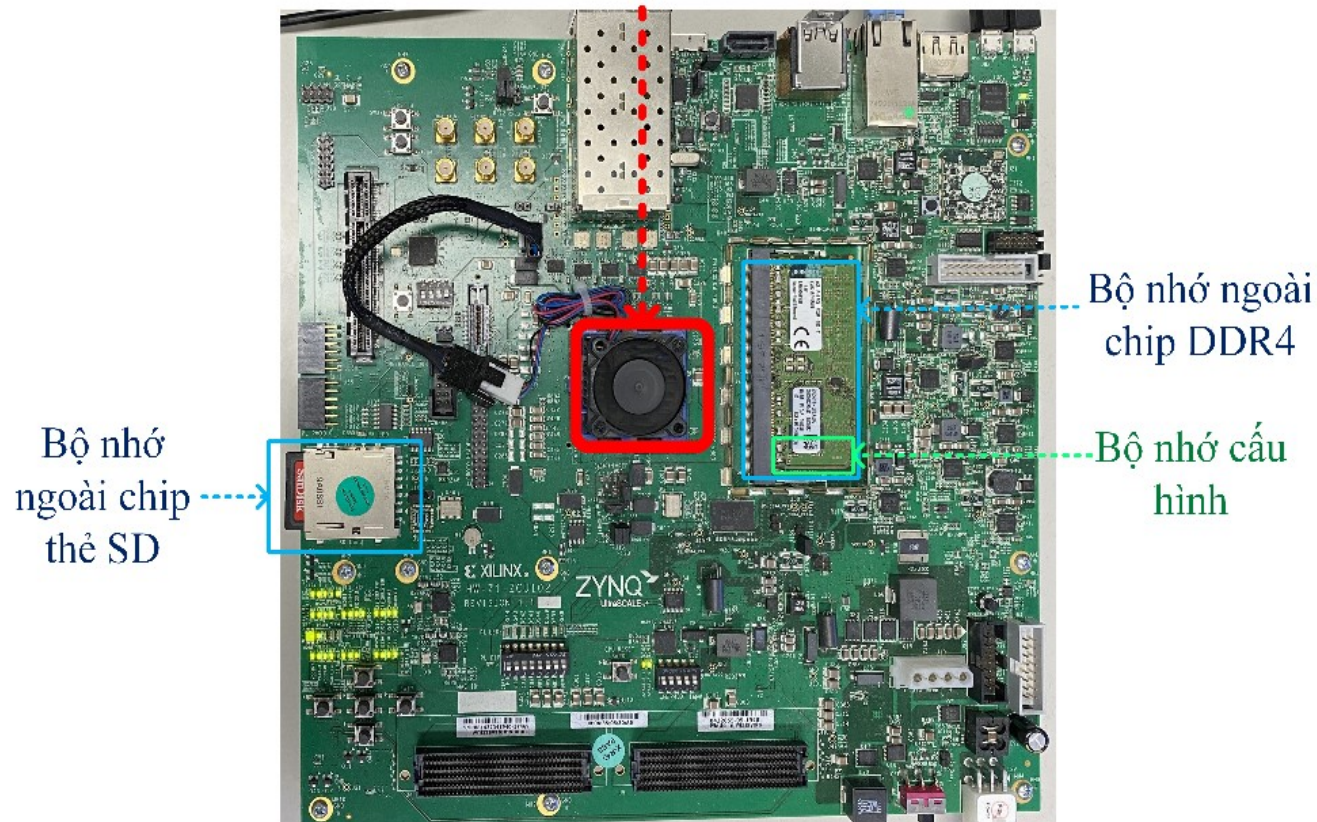
TỔNG QUAN



Kiến trúc tổng quan một SoC trên FPGA gồm:

- ✓ *Bộ nhớ trên chip (on-chip memory)*
- ✓ *Bộ nhớ ngoài chip (off-chip memory)*
- ✓ *Bộ nhớ cấu hình (configuration memory)*

Bộ nhớ trên chip URAM/BRAM/FIFO/LUTRAM
trong Mạng logic lập trình được (PL)



TỔNG QUAN

■ On chip mem:

- ✓ Bộ nhớ trên chip là các bộ nhớ tích hợp trực tiếp trong phần logic của FPGA như BRAM, LUTRAM và URAM, bộ đệm (Cache).
- ✓ Cung cấp khả năng truy cập dữ liệu nhanh với độ trễ thấp, nhờ nằm ngay trên mạch logic.
- ✓ Được sử dụng cho các tác vụ yêu cầu tốc độ xử lý cao và thời gian thực, như xử lý tín hiệu số (DSP), mã hóa/giải mã dữ liệu, và các thuật toán trí tuệ nhân tạo cần độ trễ thấp.
- ✓ Có tốc độ truy cập nhanh và băng thông cao, giúp giảm thời gian trễ giao tiếp và tăng hiệu suất.

TỔNG QUAN

■ Off chip mem:

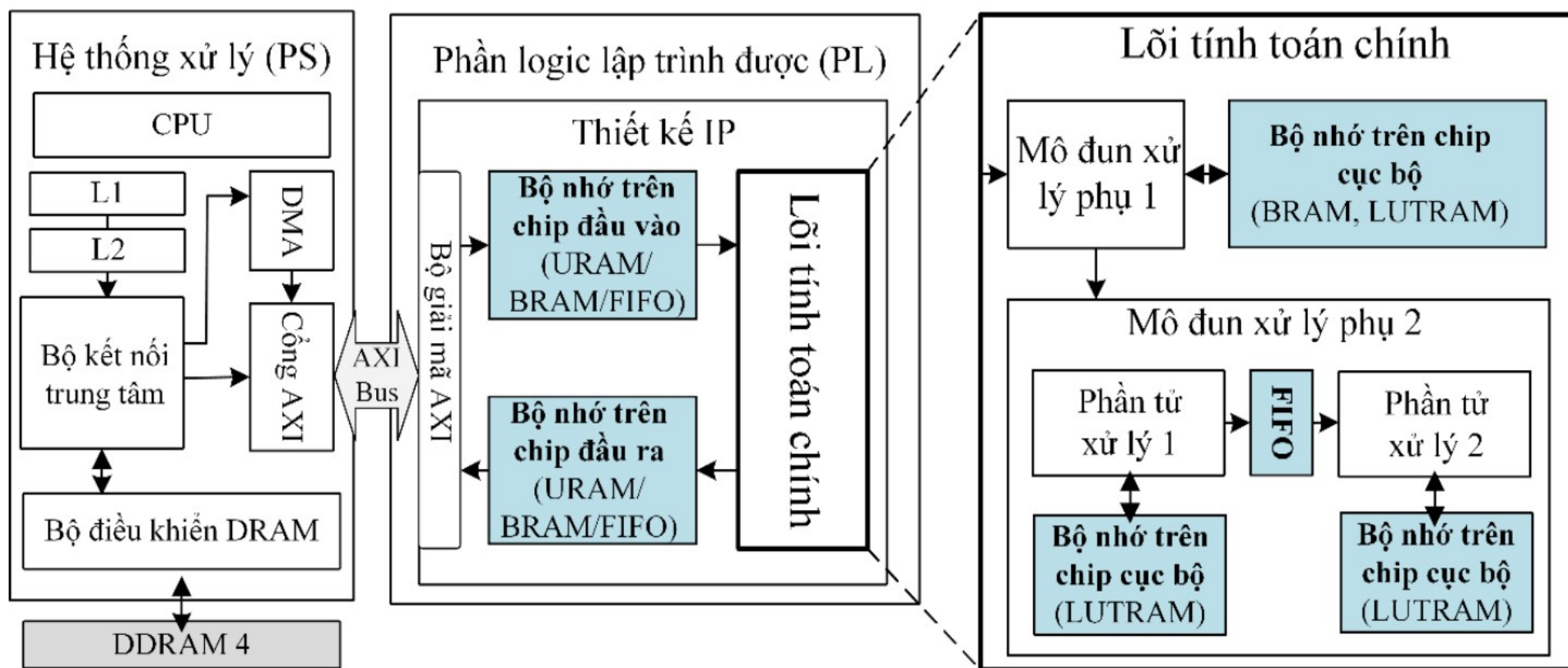
- ✓ Bộ nhớ ngoài chip bao gồm các loại bộ nhớ như SRAM, DRAM, eDRAM, ROM/PROM và các thiết bị lưu trữ như HDD/SSD và NAND Flash.
- ✓ Chúng có dung lượng lớn hơn và thường kết nối với FPGA qua các giao diện như AXI hoặc PCIe.
- ✓ Được sử dụng để lưu trữ dữ liệu khối lớn, chương trình thực thi và các tập dữ liệu cần xử lý.
- ✓ Mặc dù có độ trễ cao hơn do phải qua giao diện, bộ nhớ ngoài chip cung cấp dung lượng lớn và linh hoạt cho các ứng dụng như học máy và phân tích dữ liệu lớn.

TỔNG QUAN

■ Configuration mem:

- ✓ Bộ nhớ cấu hình thường là Flash Memory hoặc EEPROM
- ✓ Bộ nhớ cấu hình lưu trữ bitstream cho FPGA, cho phép cấu hình lại phần logic lập trình được (PL) khi FPGA khởi động hoặc thay đổi thiết kế.
- ✓ Bộ nhớ này đảm bảo FPGA có thể tải lại các thiết kế phần cứng cần thiết khi bật hoặc tái cấu hình.

BỘ NHỚ TRÊN CHIP

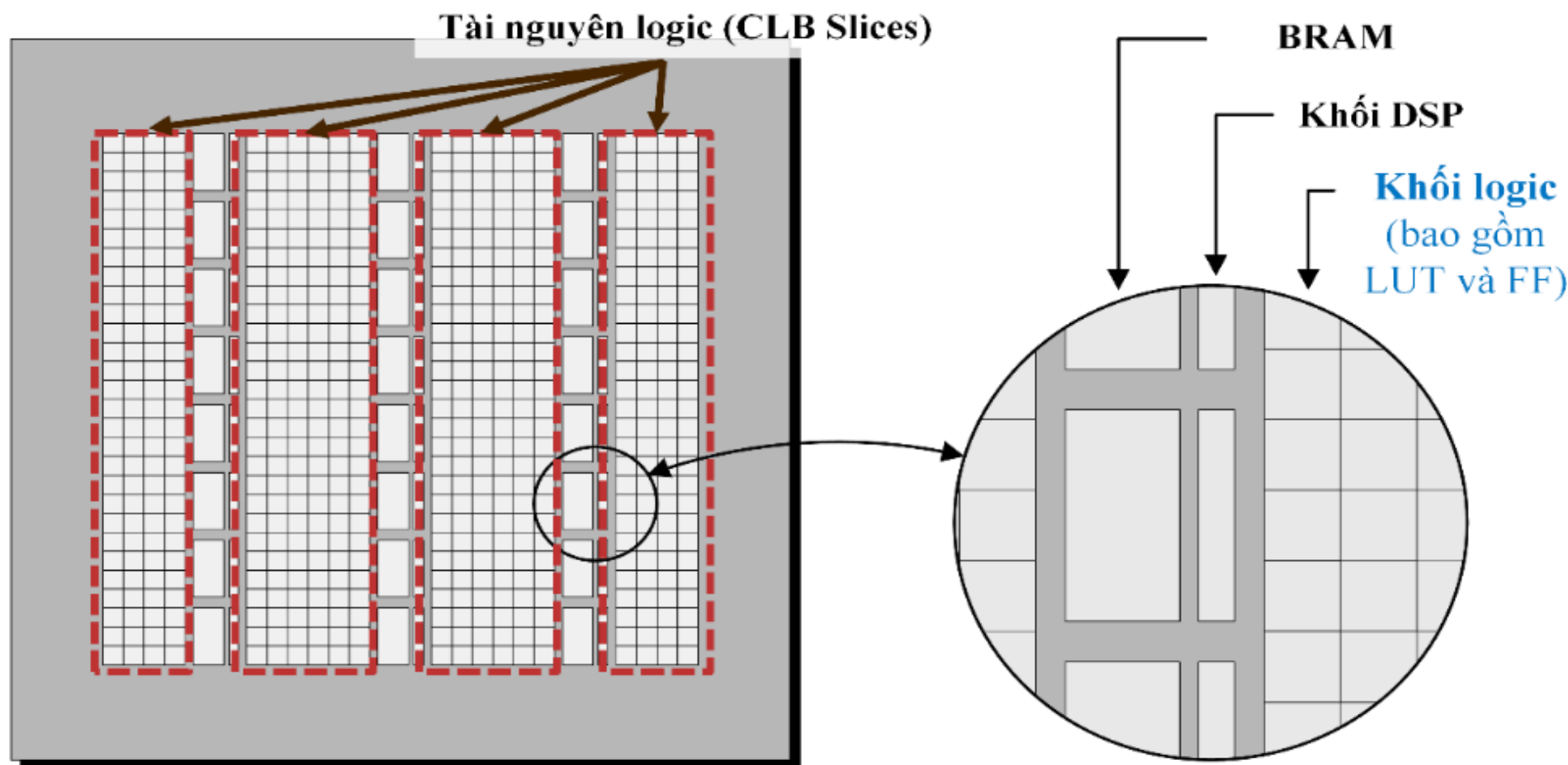


Một SoC đơn giản với thiết kế IP trên phần logic lập trình được sử dụng nhiều loại bộ nhớ trên chip để lưu trữ dữ liệu tính toán.

BỘ NHỚ TRÊN CHIP

- Các loại bộ nhớ trên chip như BRAM, URAM, LUTRAM, Cache, và FIFO được sử dụng để lưu trữ dữ liệu từ bộ nhớ ngoài chip (như DRAM) và dữ liệu nội bộ cần thiết cho các lõi IP để tính toán
- BRAM và URAM được dùng để lưu trữ dữ liệu đầu vào từ hệ thống PS hoặc bộ nhớ ngoài, cung cấp không gian lưu trữ tạm thời cho dữ liệu cần xử lý.
- BRAM hoặc LUTRAM được dùng để lưu trữ dữ liệu tạm thời.
⇒ Giảm thời gian truy xuất và tăng tốc độ xử lý
- Bộ nhớ đầu ra sử dụng FIFO để đảm bảo đồng bộ hóa và lưu trữ dữ liệu trước khi truyền ra ngoài hệ thống.

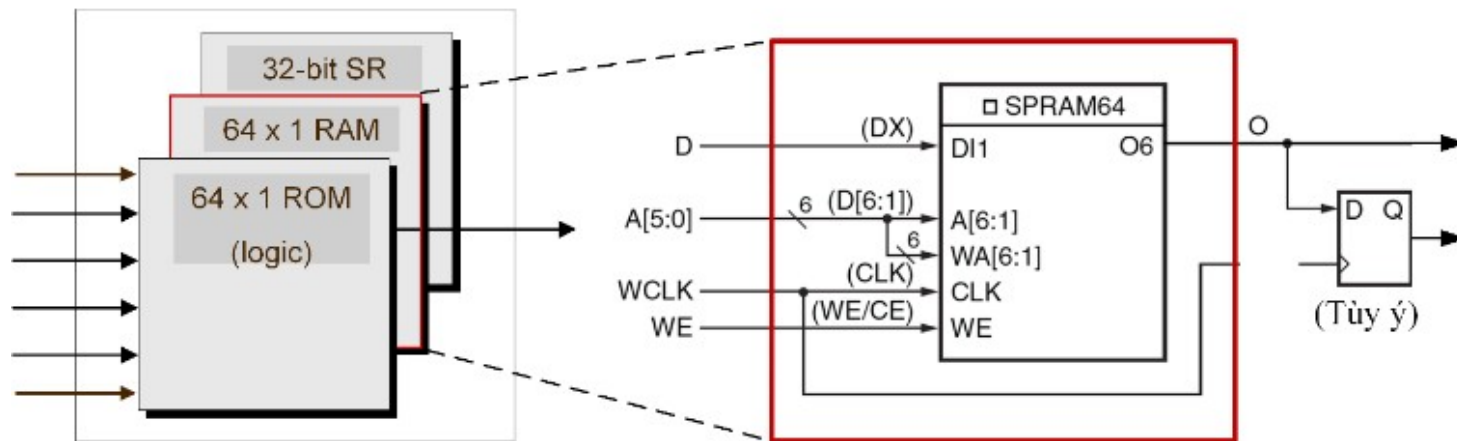
LUT RAM / Distributed RAM



Vị trí LUT trong khối logic FPGA có thể tạo thành LUTRAM.

LUT RAM / Distributed RAM

- ✓ LUTRAM, hay Distributed RAM, là bộ nhớ được cấu hình từ các Look-Up Table (LUT) trong FPGA.
- ✓ Chủ yếu dùng để lưu trữ dữ liệu nhỏ với độ trễ rất thấp.
- ✓ Phù hợp cho các bộ đệm (buffers) và bộ nhớ tạm thời (scratchpad memory). Thường được sử dụng trong máy trạng thái (state machine) để lưu trữ trạng thái.
- ✓ Các LUT nằm trong tài nguyên logic CLB (Configurable Logic Blocks) có thể tạo ra bộ nhớ nhỏ với độ trễ thấp.



Cấu trúc bên trong LUT khi được cấu hình thành bộ nhớ LUTRAM.

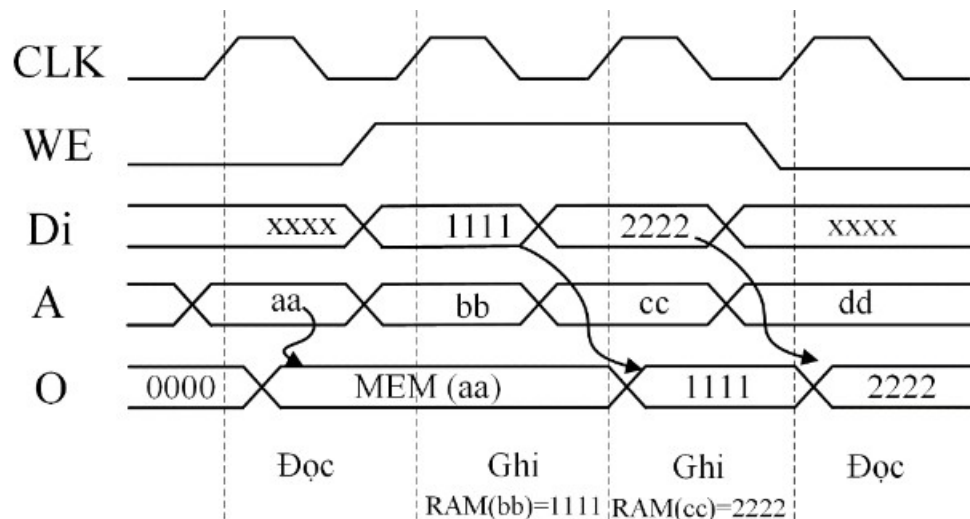
```
module LURAM (CLK, WE, A, Di, O);
input CLK;
input WE;
input [5:0] A;
input [15:0] Di;
output [15:0] O;

reg [15:0] ram [63:0];

always @(posedge CLK)
begin
if (WE)
ram[A] <= Di;
end

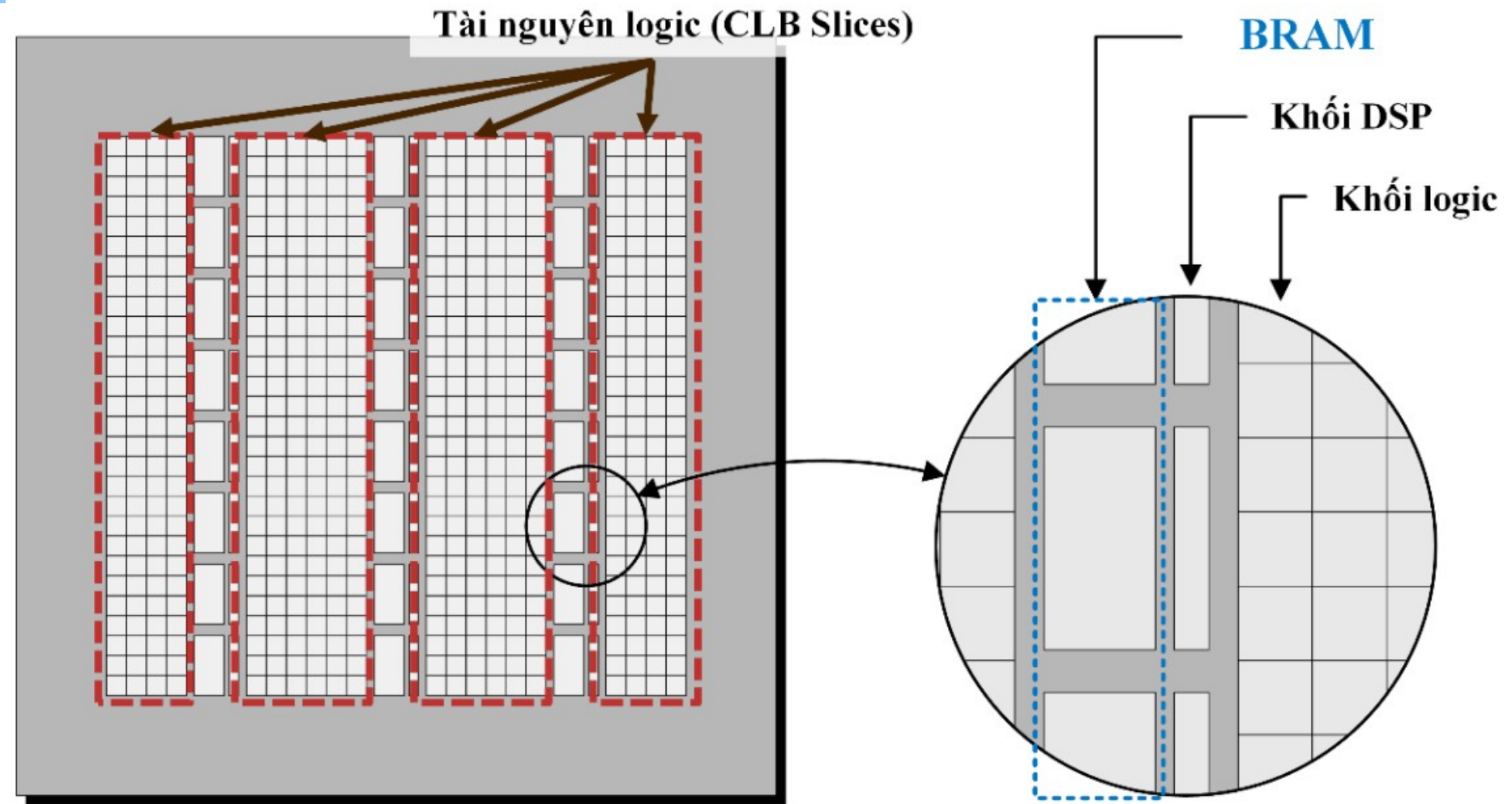
assign O = ram[A];
endmodule
```

Mô tả LUTRAM bằng Verilog.



Sơ đồ thời gian chi tiết của tín hiệu trong LUT cho việc đọc và ghi.

BLOCK RAM (BRAM)

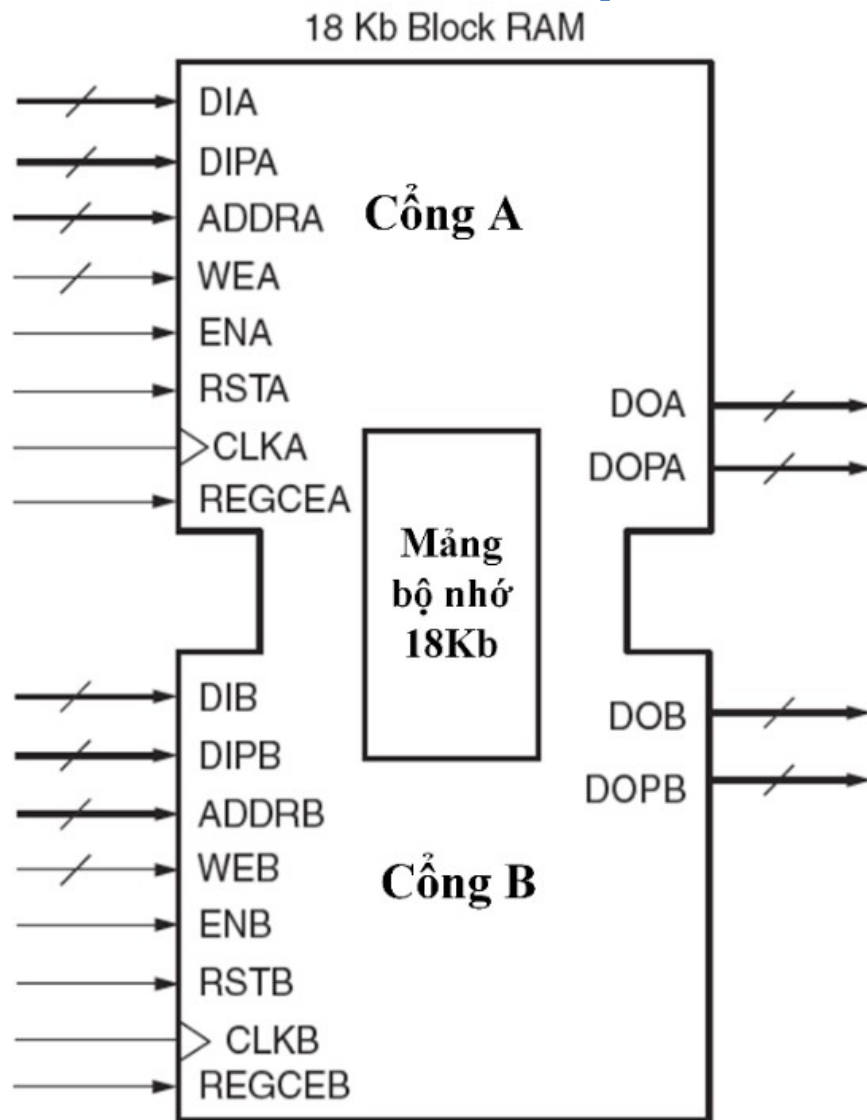


Vị trí BRAM trong FPGA.

BLOCK RAM (BRAM)

- Bộ nhớ Block RAM (BRAM) là một thành phần quan trọng trong FPGA:
 - ✓ Cung cấp khả năng lưu trữ dữ liệu trực tiếp trên chip với độ trễ thấp và băng thông cao.
 - ✓ Kích thước lớn hơn LUTRAM → sử dụng để lưu trữ các khối dữ liệu lớn hơn.
 - ✓ Cấu trúc BRAM là các khối bộ nhớ độc lập, cho phép thiết kế linh hoạt trong lưu trữ và truy xuất dữ liệu.
 - ✓ Ứng dụng cho các yêu cầu tốc độ xử lý cao, như xử lý tín hiệu số và điều khiển hệ thống.
 - ✓ Trong thiết kế IP, BRAM có thể là bộ nhớ toàn cục để lưu trữ dữ liệu lớn hoặc bộ nhớ cục bộ cho dữ liệu tạm thời.

BLOCK RAM (BRAM)

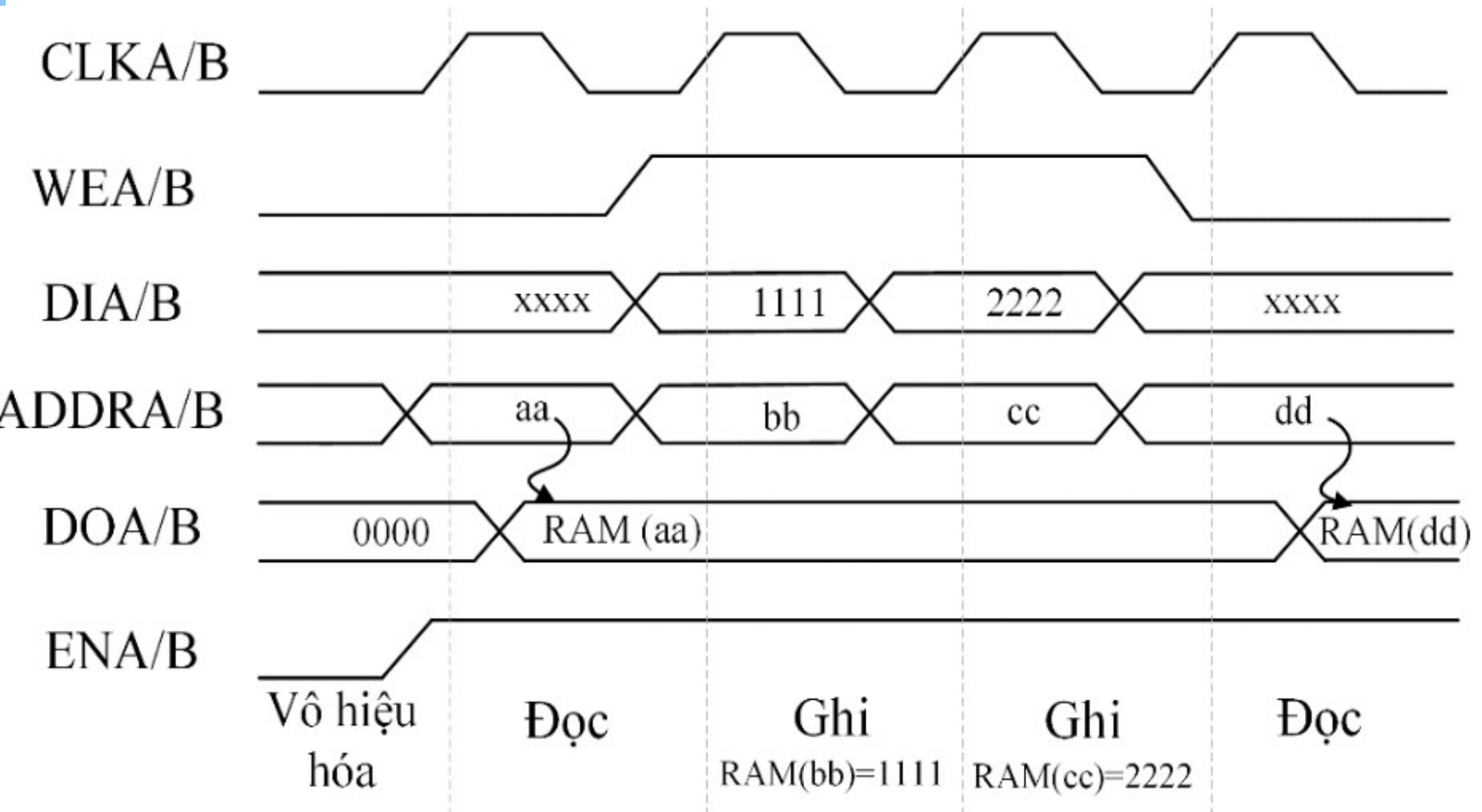


Giao diện bộ nhớ BRAM.

BLOCK RAM (BRAM)

Tên tín hiệu của cổng A và B	Mô tả
DIA/B	Tín hiệu đầu vào dữ liệu
DIPA/B	Tín hiệu đầu vào kiểm tra chẵn lẻ dữ liệu
ADDRA/B	Tín hiệu địa chỉ
WEA/B	Cho phép ghi theo chiều rộng byte
ENA/B	Khi không hoạt động, không có dữ liệu nào được ghi vào RAM khối và bus đầu ra vẫn giữ nguyên trạng thái trước đó.
RSTA/B	Đặt lại/Thiết lập đồng bộ các thanh ghi đầu ra (DO_REG = 1).
CLKA/B	Đầu vào xung nhịp
DOA/B	Tín hiệu đầu ra dữ liệu
DOPA/B	Tín hiệu đầu ra kiểm tra chẵn lẻ dữ liệu
REGCEA/B	Cho phép xung nhịp thanh ghi đầu ra

BLOCK RAM (BRAM)

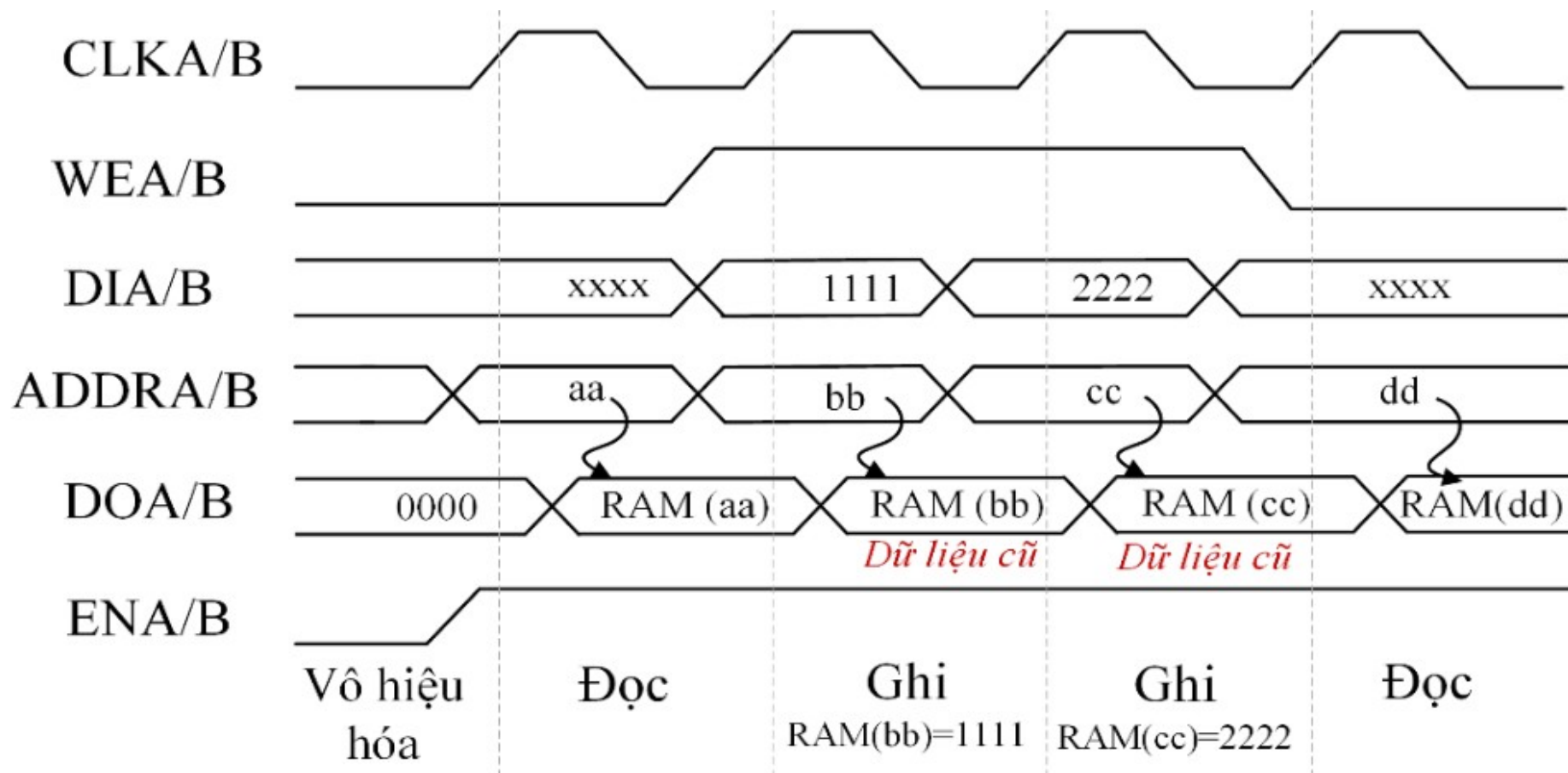


Sơ đồ thời gian chi tiết của tín hiệu trong BRAM cho việc đọc và ghi dữ liệu ở chế độ NO_CHANGE.

BLOCK RAM (BRAM)

- Chế độ NO_CHANGE: Giá trị đầu ra không thay đổi trong quá trình ghi
 - Khi có tín hiệu đồng thời đọc và ghi → tín hiệu dữ liệu đầu ra (DOA/DOA) giữ nguyên giá trị trước đó
 - Khi WEA/B được kích hoạt, dữ liệu mới sẽ được ghi vào địa chỉ tương ứng (ADDRA/B), nhưng DOA/B vẫn giữ giá trị cũ cho đến khi việc ghi hoàn tất.
- ⇒ NO_CHANGE duy trì tính ổn định của dữ liệu đầu ra

BLOCK RAM (BRAM)

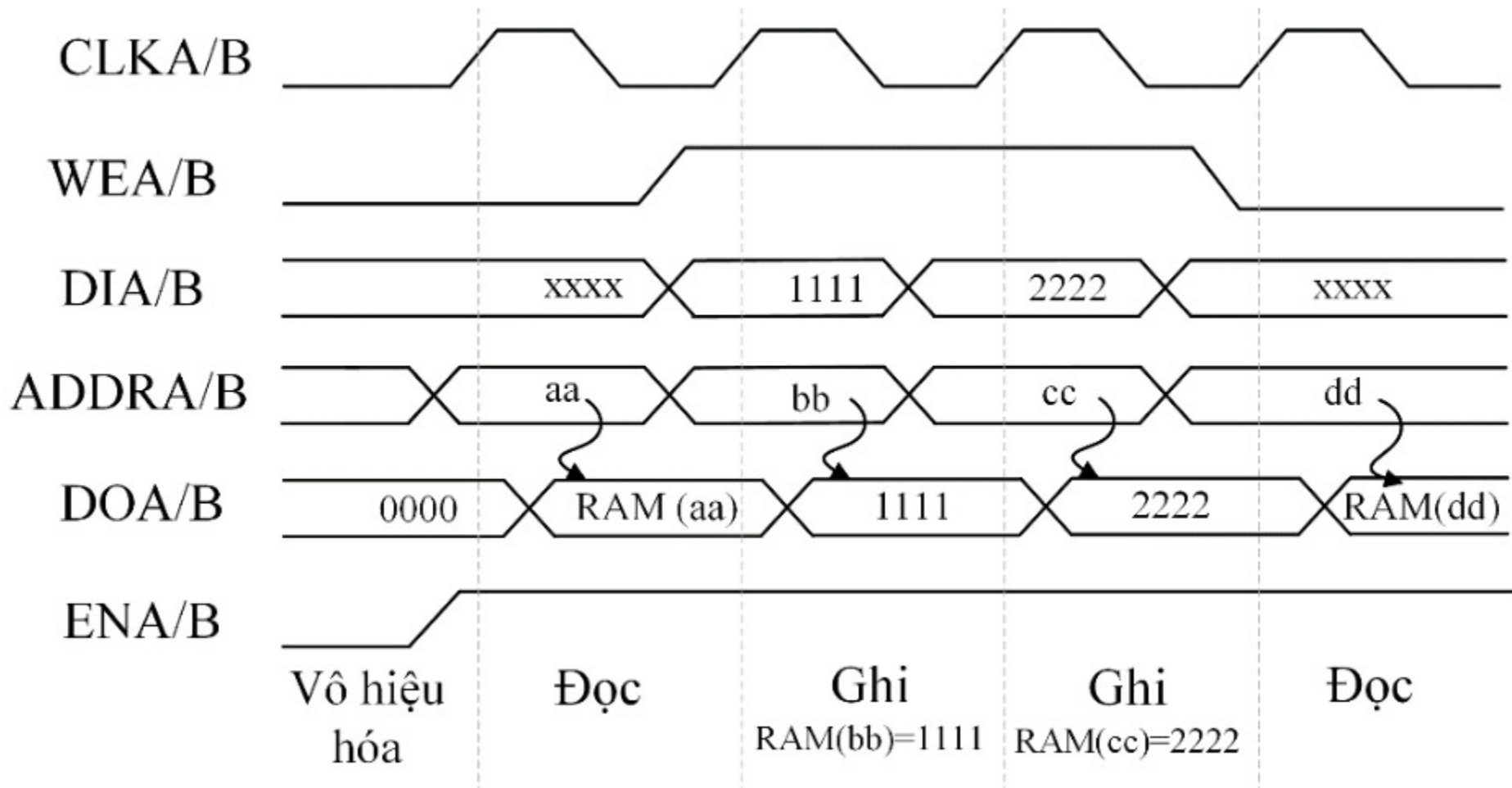


Sơ đồ thời gian chi tiết của tín hiệu trong BRAM cho việc đọc và ghi dữ liệu ở chế độ *READ_FIRST*.

BLOCK RAM (BRAM)

- Chế độ READ_FIRST: Giá trị đầu ra luôn phản ánh giá trị trước khi cập nhật
 - Khi có truy cập đồng thời đọc và ghi, việc đọc dữ liệu sẽ được thực hiện trước, dữ liệu được đọc tại thời điểm truy xuất và sau đó mới ghi dữ liệu mới vào bộ nhớ
 - Khi WEA/B được kích hoạt, bộ nhớ ghi tại địa chỉ ADDRA/B, nhưng DIA/B chỉ được ghi sau khi việc đọc hoàn tất.
- ⇒ READ_FIRST đảm bảo rằng dữ liệu được đọc chính xác trước khi thực hiện bất kỳ thay đổi nào trong bộ nhớ.

BLOCK RAM (BRAM)



Sơ đồ thời gian chi tiết của tín hiệu trong BRAM cho việc đọc và ghi dữ liệu ở chế độ WRITE_FIRST.

BLOCK RAM (BRAM)

- Chế độ WRITE_FIRST: Giá trị đầu vào xuất hiện ngay lập tức tại đầu ra.
 - Dữ liệu mới được ghi trực tiếp vào bộ nhớ và xuất hiện ngay lập tức tại đầu ra, không phụ thuộc vào truy cập đọc đồng thời.
 - Dữ liệu mới được cập nhật ngay lập tức tại đầu ra (DOA/B) mà không cần chờ hoàn tất quá trình đọc.
- ⇒ WRITE_FIRST thường được sử dụng khi cần cập nhật dữ liệu nhanh chóng, đặc biệt trong các ứng dụng yêu cầu phản ánh ngay lập tức dữ liệu mới.

Code Verilog mô tả Dual-Port BRAM với hai cổng đọc và ghi, sử dụng bộ nhớ BRAM trong thiết kế IP.

```
module BRAM #(
    parameter AWIDTH = 10, // độ rộng địa chỉ
    parameter DWIDTH = 32 // độ rộng dữ liệu
)(input clka, // xung nhịp (clock)
   //**** Cổng A ****//
   input ena, // tín hiệu cho phép đọc cổng A
   input wea, // tín hiệu cho phép ghi cổng A
   input [AWIDTH-1:0] addra, // địa chỉ cổng A
   input [DWIDTH-1:0] dina, // dữ liệu cổng A
   output reg [DWIDTH-1:0] douta, // dữ liệu đầu ra cổng A
   //**** Cổng B ****//
   input clkb, // xung nhịp (clock)
   input enb, // tín hiệu cho phép đọc cổng B
   input web, // tín hiệu cho phép ghi cổng B
   input [AWIDTH-1:0] addrb, // địa chỉ cổng B
   input [DWIDTH-1:0] dinb, // dữ liệu cổng B
   output reg [DWIDTH-1:0] doutb ); // dữ liệu đầu ra cổng B
// Khai báo bộ nhớ BRAM với kiểu "block",
// sử dụng kích thước bộ nhớ theo độ rộng địa chỉ AWIDTH và độ rộng dữ liệu DWIDTH.
(* ram_style = "block" *) reg [DWIDTH-1:0] mem [2**AWIDTH-1:0];

always @(posedge clka) begin
    // **** Cổng A ****//
    if (ena) begin
        if(wea) begin
            mem[addra] <= dina; // ghi dữ liệu vào bộ nhớ tại địa chỉ addra
        end
        douta <= mem[addra]; // đọc dữ liệu từ bộ nhớ tại địa chỉ addra
    end
end

always @(posedge clkb) begin
    // **** Cổng B ****//
    if (enb) begin
        if(web) begin
            mem[addrb] <= dinb; // ghi dữ liệu vào bộ nhớ tại địa chỉ addrb
        end
        doutb <= mem[addrb]; // đọc dữ liệu từ bộ nhớ tại địa chỉ addrb
    end
end
endmodule
```

Code Verilog để thay đổi chế độ của BRAM với các chế độ NO_CHANGE, READ_FIRST, và WRITE_FIRST

```
/* 1. Chế độ NO_CHANGE:*/
```

```
always @(posedge clka) begin
    // /*** Cổng A ***/
    if (ena) begin
        if (wea) begin
            mem[addra] <= dina; // ghi dữ liệu vào bộ nhớ tại địa chỉ addra
        end
        douta <= douta; // giữ nguyên giá trị của douta trong suốt quá trình ghi
    end
end
```

```
/* 2. Chế độ READ_FIRST:*/
```

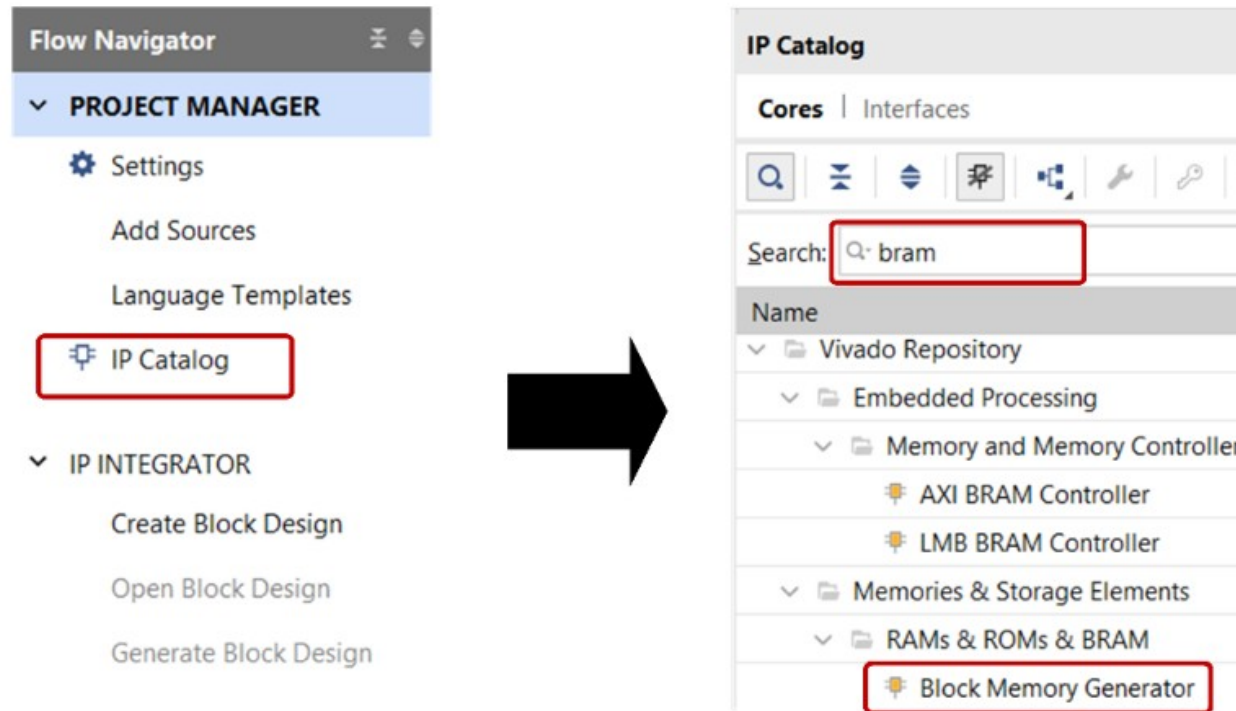
```
always @(posedge clka) begin
    // /*** Cổng A ***/
    if (ena) begin
        douta <= mem[addra]; // đọc dữ liệu từ bộ nhớ tại địa chỉ addra
        if (wea) begin
            mem[addra] <= dina; // ghi dữ liệu vào bộ nhớ tại địa chỉ addra
        end
    end
end
```

```
/* 3. Chế độ WRITE_FIRST:*/
```

```
always @(posedge clka) begin
    // /*** Cổng A ***/
    if (ena) begin
        if (wea) begin
            mem[addra] <= dina; // ghi dữ liệu vào bộ nhớ tại địa chỉ addra
            douta <= dina; // cập nhật ngay lập tức dữ liệu đầu ra sau khi ghi
        end
        else begin
            douta <= mem[addra]; // đọc dữ liệu từ bộ nhớ tại địa chỉ addra
        end
    end
end
```

BLOCK RAM (BRAM)

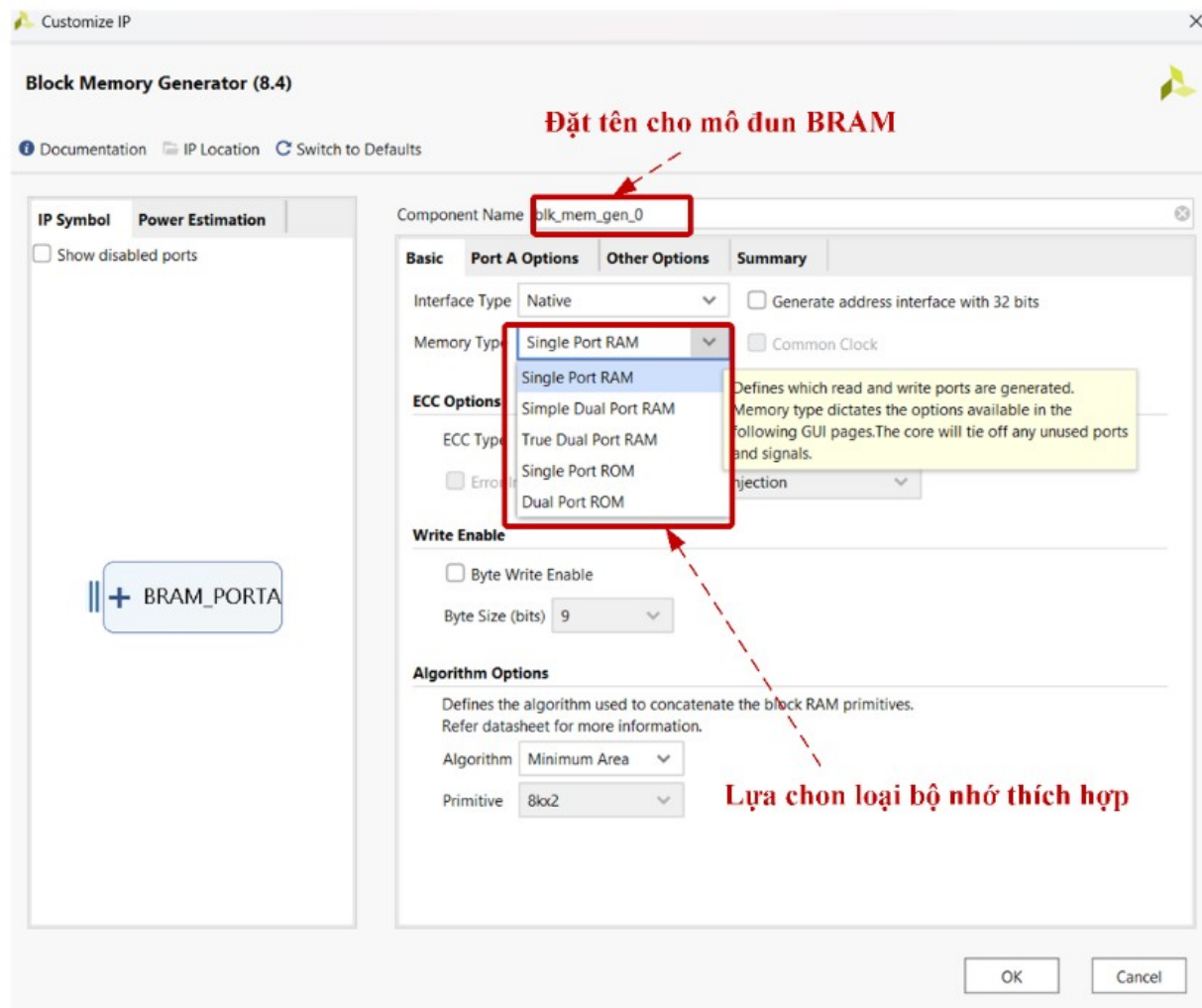
■ Gọi mô-đun IP BRAM có sẵn :



Giao diện IP Catalog trong Vivado, nơi tìm kiếm và chọn mô-đun Block Memory Generator để tạo và cấu hình BRAM trong thiết kế IP ở FPGA.

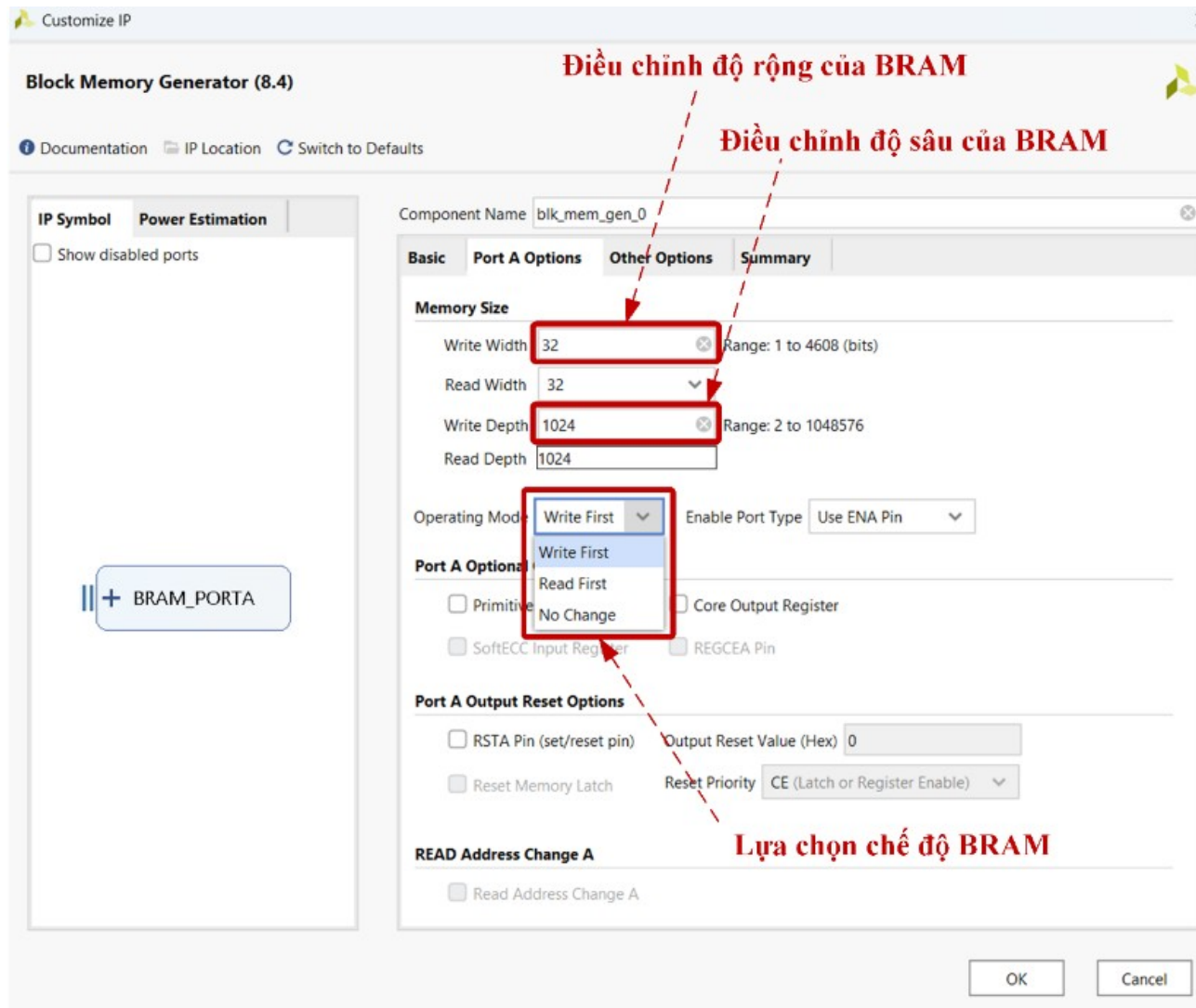
- Người thiết kế có thể mở IP Catalog, tìm kiếm và chọn mô-đun Block Memory Generator, sau đó cấu hình các tham số của BRAM như kích thước bộ nhớ, chế độ đọc và ghi, và số cổng cần thiết cho ứng dụng cụ thể

BLOCK RAM (BRAM)



Giao diện cấu hình mô-đun Block Memory Generator trong Vivado, nơi người thiết kế có thể đặt tên cho mô-đun BRAM và lựa chọn loại bộ nhớ thích hợp.

BLOCK RAM (BRAM)



Giao diện cấu hình mô-đun Block Memory Generator trong Vivado, nơi người thiết kế điều chỉnh độ rộng, độ sâu bộ nhớ BRAM và chọn chế độ hoạt động.

BLOCK RAM (BRAM)

PROJECT MANAGER - Simulation **Tên các cổng input và output của IP BRAM để các mô-đun khác gọi đến**

Sources

Design Sources (7)

Non-module Files (6)

BRAM (BRAM.xci) (1)

BRAM(BRAM_arch) (BRAM.vhd) (1)

U0: blk_mem_gen_v8_4_4(xilinx)

Constraints (1)

Simulation Sources (8)

Utility Sources

BRAM.vhd

e:/Research/FQsun/Simulation/Simulation.gen/sources_1/ip/BRAM/synth/BRAM.vhd

```

59 ENTITY BRAM IS
60   PORT (
61     clka : IN STD_LOGIC;
62     ena : IN STD_LOGIC;
63     wea : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
64     addra : IN STD_LOGIC_VECTOR(9 DOWNTO 0);
65     dina : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
66     douta : OUT STD_LOGIC_VECTOR(31 DOWNTO 0);
67     clkfb : IN STD_LOGIC;
68     enb : IN STD_LOGIC;
69     web : IN STD_LOGIC_VECTOR(0 DOWNTO 0);
70     addrb : IN STD_LOGIC_VECTOR(9 DOWNTO 0);
71     dinb : IN STD_LOGIC_VECTOR(31 DOWNTO 0);
72     doutb : OUT STD_LOGIC_VECTOR(31 DOWNTO 0);
73   );
74 END BRAM;
        
```

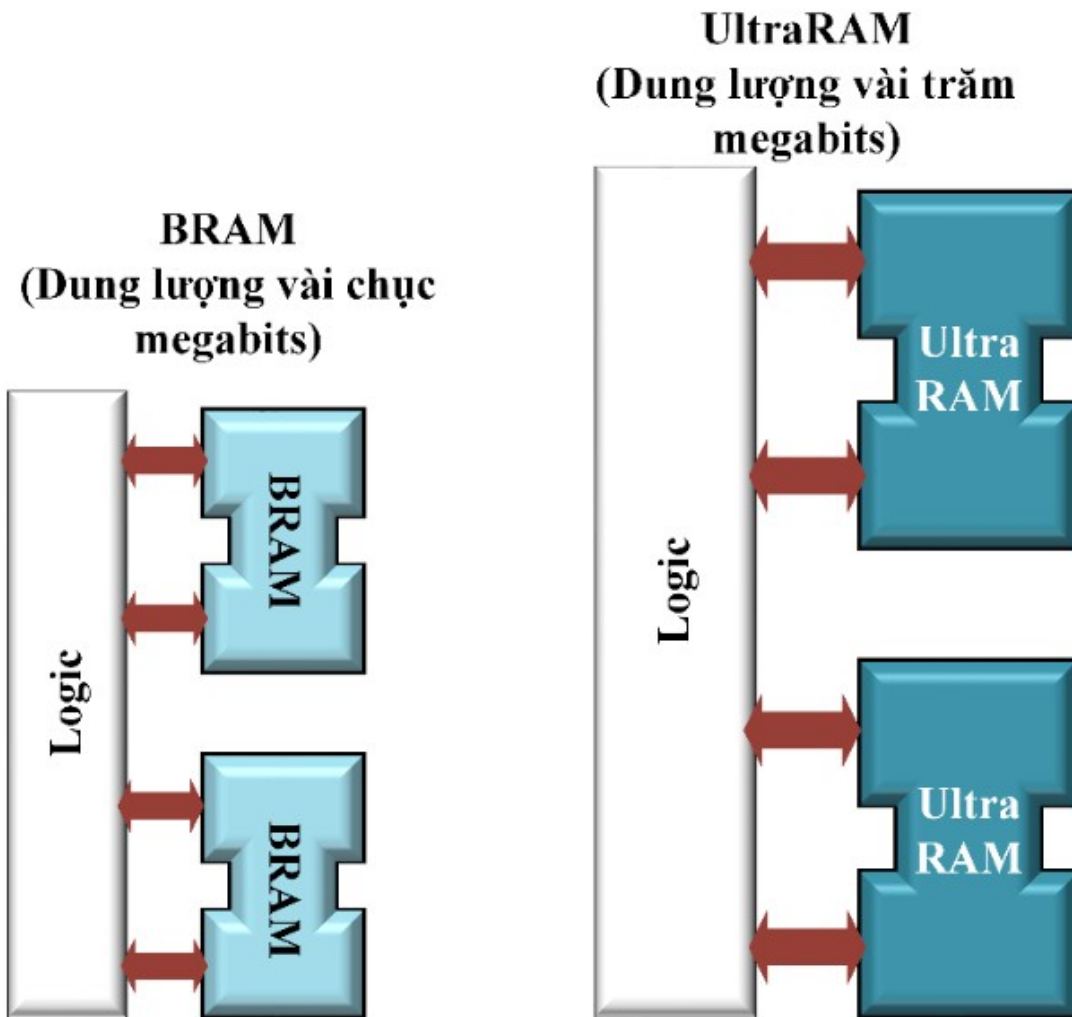
IP BRAM được gọi sẽ hiển thị trong tab Design Sources

Giao diện hiển thị các cổng input và output của IP BRAM trong tab Design Sources, nơi mô-đun BRAM được gọi để các mô-đun khác có thể kết nối và sử dụng.

ULTRA RAM

- ✓ URAM có dung lượng lớn hơn BRAM, cung cấp băng thông cao hơn và độ trễ thấp hơn.
- ✓ URAM tiết kiệm năng lượng tốt hơn nhờ tính năng "auto sleep".
- ✓ URAM không gặp phải va chạm địa chỉ, tăng cường sự ổn định trong xử lý dữ liệu.
- ✓ BRAM hỗ trợ các chế độ độc lập như đọc-ghi đồng thời (TDP) và tuần tự (SDP), trong khi URAM chỉ cho phép một thao tác đọc hoặc ghi mỗi chu kỳ.
- ✓ URAM không có khả năng điều chỉnh chế độ đọc (read-first, write-first) và không hỗ trợ bộ điều khiển đầu vào/ra động.
- ✓ Yêu cầu độ rộng dữ liệu phải là 72 bit và độ rộng địa chỉ phải từ 2^{12} (hoặc 12 bit) trở lên để tối ưu hiệu suất sử dụng

ULTRA RAM



So sánh dung lượng giữa BRAM và URAM trong FPGA.

ULTRA RAM

```

module URAM #(
    parameter AWIDTH = 12, // độ rộng địa chỉ
    parameter DWIDTH = 72 // độ rộng dữ liệu
)(
    input clk,                // Clock
    input ena,                // Tín hiệu cho phép đọc cổng A
    input wea,                // Tín hiệu cho phép ghi cổng A
    input [AWIDTH-1:0] addra,  // Địa chỉ cổng A
    input [DWIDTH-1:0] dina,   // Dữ liệu cổng A
    output reg [DWIDTH-1:0] douta // Dữ liệu đầu ra cổng A
);

// Khai báo bộ nhớ URAM với kiểu "ultra",
// sử dụng kích thước bộ nhớ theo độ rộng địa chỉ AWIDTH và độ rộng dữ liệu DWIDTH.
(* ram_style = "ultra" *) reg [DWIDTH-1:0] mem [2**AWIDTH-1:0];

// Việc ghi và đọc dữ liệu từ bộ nhớ sẽ được thực hiện với độ trễ là một chu kỳ xung nhịp
always @ (posedge clk) begin
    if (ena) begin
        if (wea) begin
            mem[addra] <= dina; // Ghi dữ liệu vào bộ nhớ tại địa chỉ addra
        end
        douta <= mem[addra]; // Đọc dữ liệu từ bộ nhớ tại địa chỉ addra
    end
end

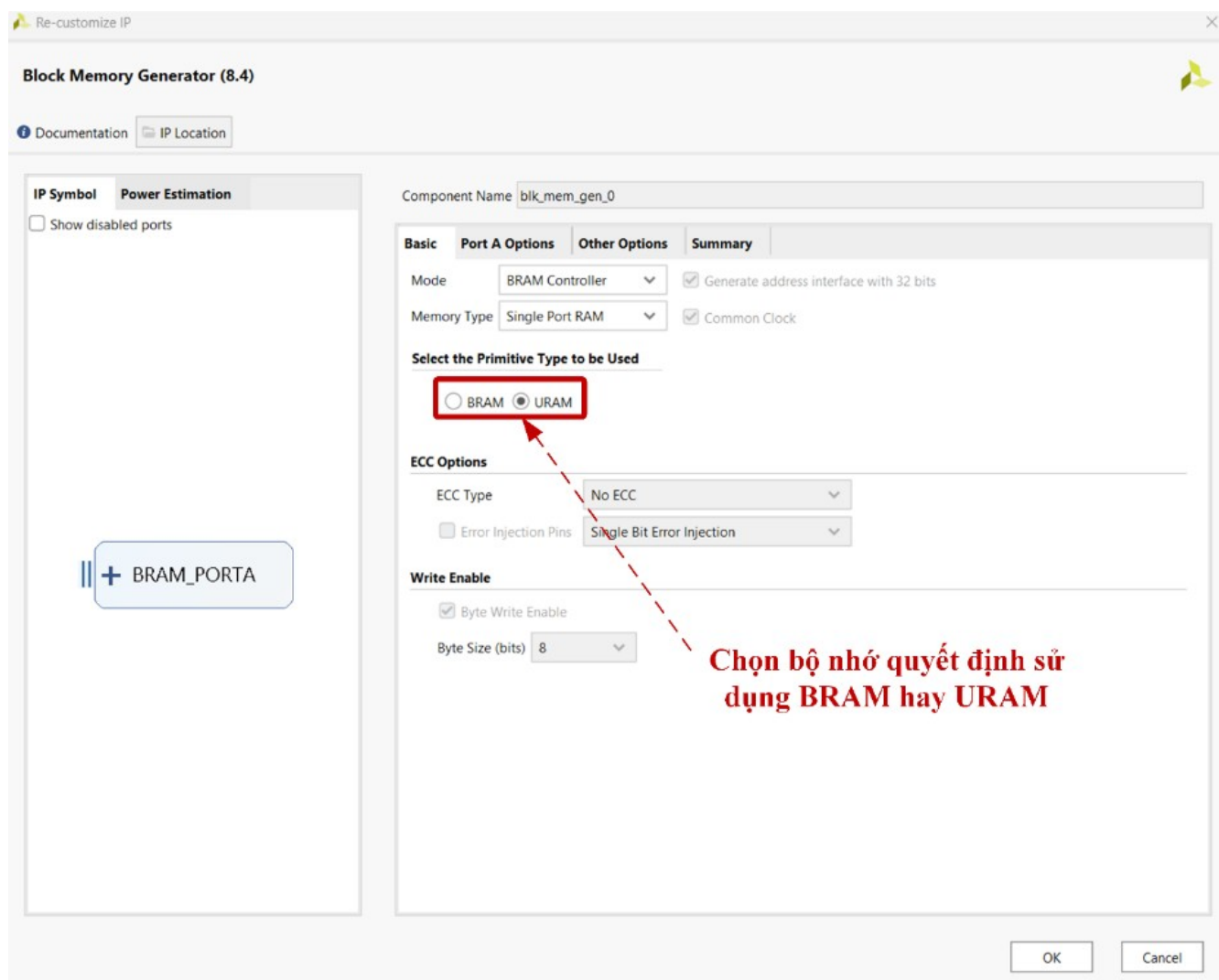
endmodule

```

Code Verilog mô tả URAM với một cổng đọc và ghi trong thiết kế IP.

ULTRA RAM

- Gọi mô-đun IP URAM có sẵn:



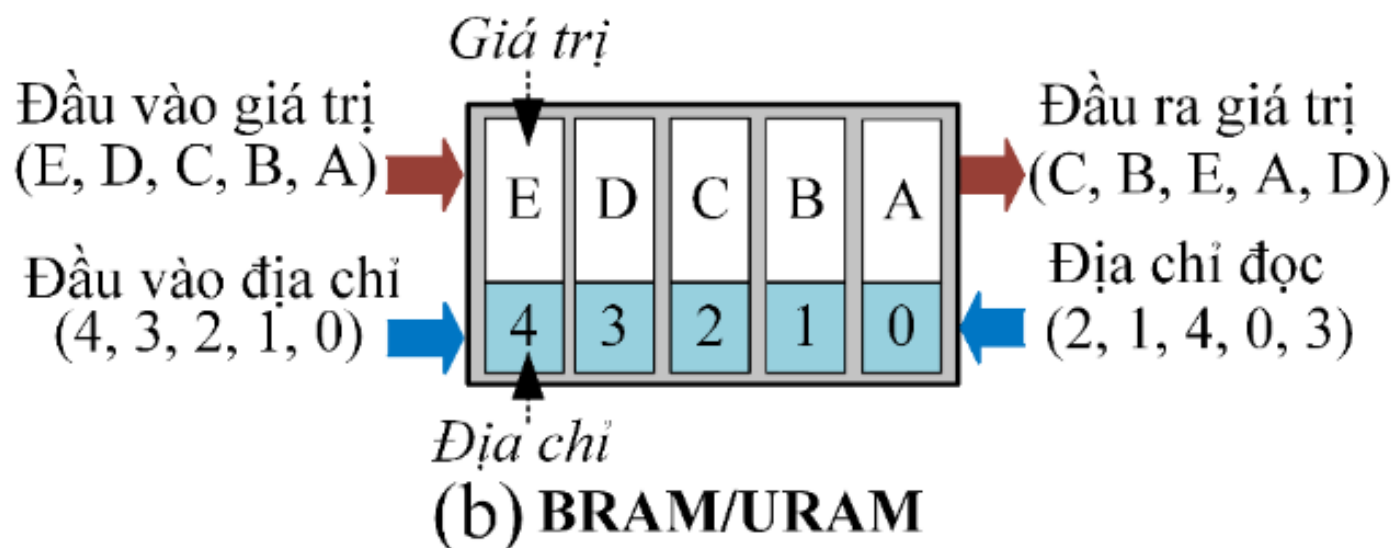
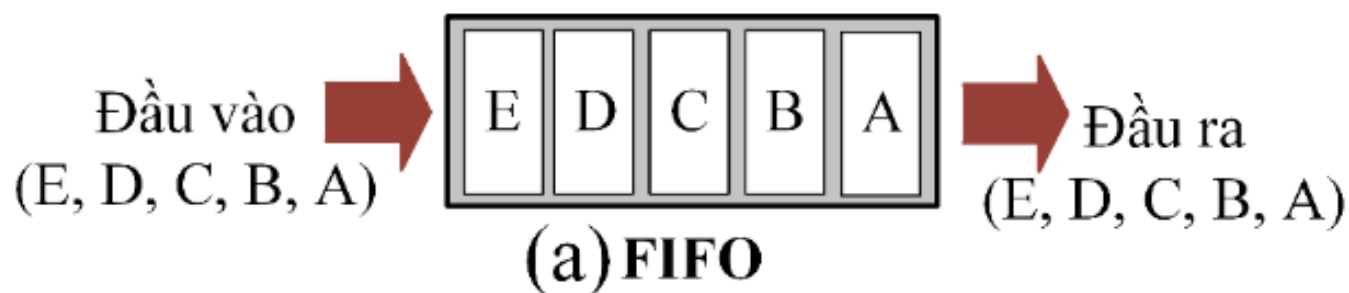
Giao diện cấu hình mô-đun Block Memory Generator trong Vivado, nơi người thiết kế điều chỉnh bộ nhớ sử dụng BRAM hay URAM.

BỘ NHỚ ĐỆM

Bộ nhớ đệm gồm: L1, L2, L3 Cache (SRAM). Bộ nhớ này được hình thành từ bộ nhớ LUTRAM, BRAM hay UltraRAM.

- L1 Cache với thời gian truy cập nhanh nhất (3-4 chu kỳ) và dung lượng khoảng 64KB, tăng tốc độ truy cập dữ liệu cho CPU.
- L2 Cache có dung lượng 256KB, trong khi L3 Cache từ 2 đến 8MB, cả hai có tốc độ truy cập chậm hơn (10-60 chu kỳ).
- Dùng cho lưu trữ tạm thời dữ liệu và lệnh thường xuyên truy cập, giúp giảm độ trễ và cải thiện hiệu suất hệ thống.

FIFO



So sánh cách lưu trữ và truy xuất dữ liệu giữa (a) FIFO và (b) BRAM/URAM.

FIFO

■ Lợi thế của FIFO:

- ✓ Quản lý thứ tự dữ liệu: FIFO xử lý theo thứ tự "vào trước, ra trước"
- ✓ Thiết kế đơn giản: Cấu trúc của FIFO giúp tiết kiệm tài nguyên bộ nhớ khi lưu trữ dữ liệu nhỏ với độ trễ thấp.
- ✓ Quản lý bộ đệm: FIFO thường được sử dụng làm bộ đệm giữa các hệ thống không đồng bộ, giúp tránh tràn bộ đệm và mất mát dữ liệu do tốc độ truyền tải không đồng đều.

■ Hạn chế của FIFO:

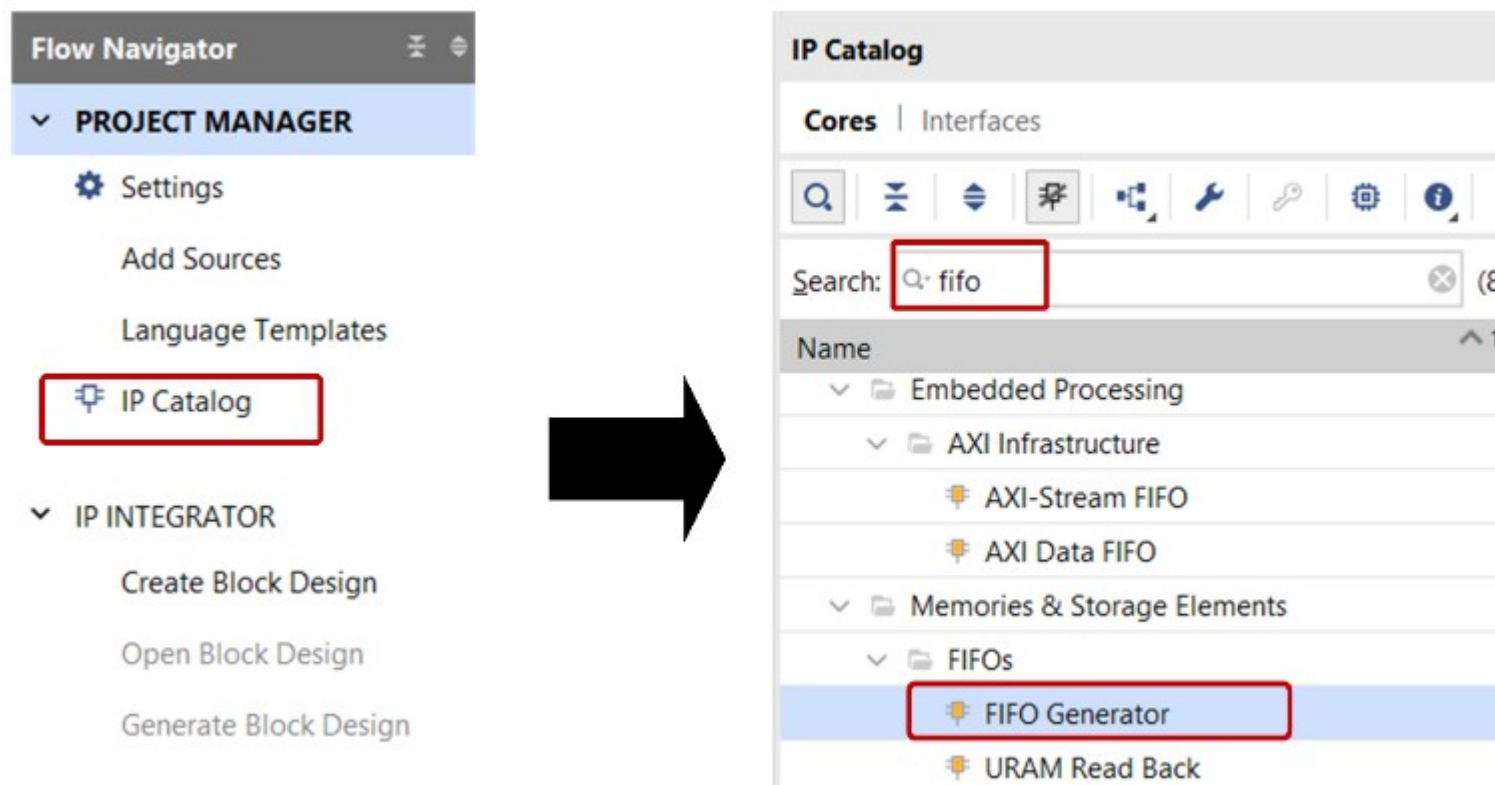
- ✓ Truy xuất dữ liệu: FIFO không hỗ trợ truy xuất ngẫu nhiên
- ✓ Dung lượng hạn chế: FIFO thường có dung lượng nhỏ hơn BRAM/URAM
- ✓ Hạn chế cổng truy cập: FIFO chỉ hỗ trợ một cổng đọc và một cổng ghi

Mã Verilog mô tả FIFO với độ rộng địa chỉ 10 bit và dữ liệu 32 bit.

```
module FIFO #(parameter AWIDTH = 10, DWIDTH = 32) (  
    input clk, srst, // Tín hiệu clock và reset  
    input wr_en, rd_en, // Tín hiệu ghi và đọc  
    input [DWIDTH-1:0] din, // Dữ liệu đầu vào  
    output reg [DWIDTH-1:0] dout, // Dữ liệu đầu ra  
    output full, empty, // Trạng thái đầy và trống  
    output wr_rst_busy, rd_rst_busy // Trạng thái reset của ghi và đọc  
);  
  
// Con trỏ đọc và viết, số bit của con trỏ là AWIDTH  
reg [AWIDTH-1:0] w_ptr, r_ptr; // Sử dụng AWIDTH để xác định số bit của con trỏ  
// FIFO sử dụng mảng 1 chiều với số phần tử DEPTH = 2^AWIDTH (với AWIDTH = 10)  
reg [DWIDTH-1:0] fifo[(1 << AWIDTH) - 1:0];  
  
always @(posedge clk or negedge srst) begin  
    if (!srst) begin  
        w_ptr <= 0;  
        r_ptr <= 0;  
        dout <= 0;  
    end else begin  
        // Ghi dữ liệu vào FIFO  
        if (wr_en && !full) begin  
            fifo[w_ptr] <= din;  
            w_ptr <= w_ptr + 1;  
        end  
  
        // Đọc dữ liệu từ FIFO  
        if (rd_en && !empty) begin  
            dout <= fifo[r_ptr];  
            r_ptr <= r_ptr + 1;  
        end  
    end  
end  
  
// Tính toán trạng thái đầy và trống  
assign full = ((w_ptr + 1) == r_ptr); // FIFO đầy khi chỉ số ghi kế tiếp trùng với chỉ số đọc  
assign empty = (w_ptr == r_ptr); // FIFO trống khi con trỏ ghi và con trỏ đọc bằng nhau  
// Tính toán trạng thái reset của ghi và đọc  
assign wr_rst_busy = (w_ptr != r_ptr); // Kiểm tra nếu FIFO đang bận ghi  
assign rd_rst_busy = (r_ptr != w_ptr); // Kiểm tra nếu FIFO đang bận đọc  
endmodule
```

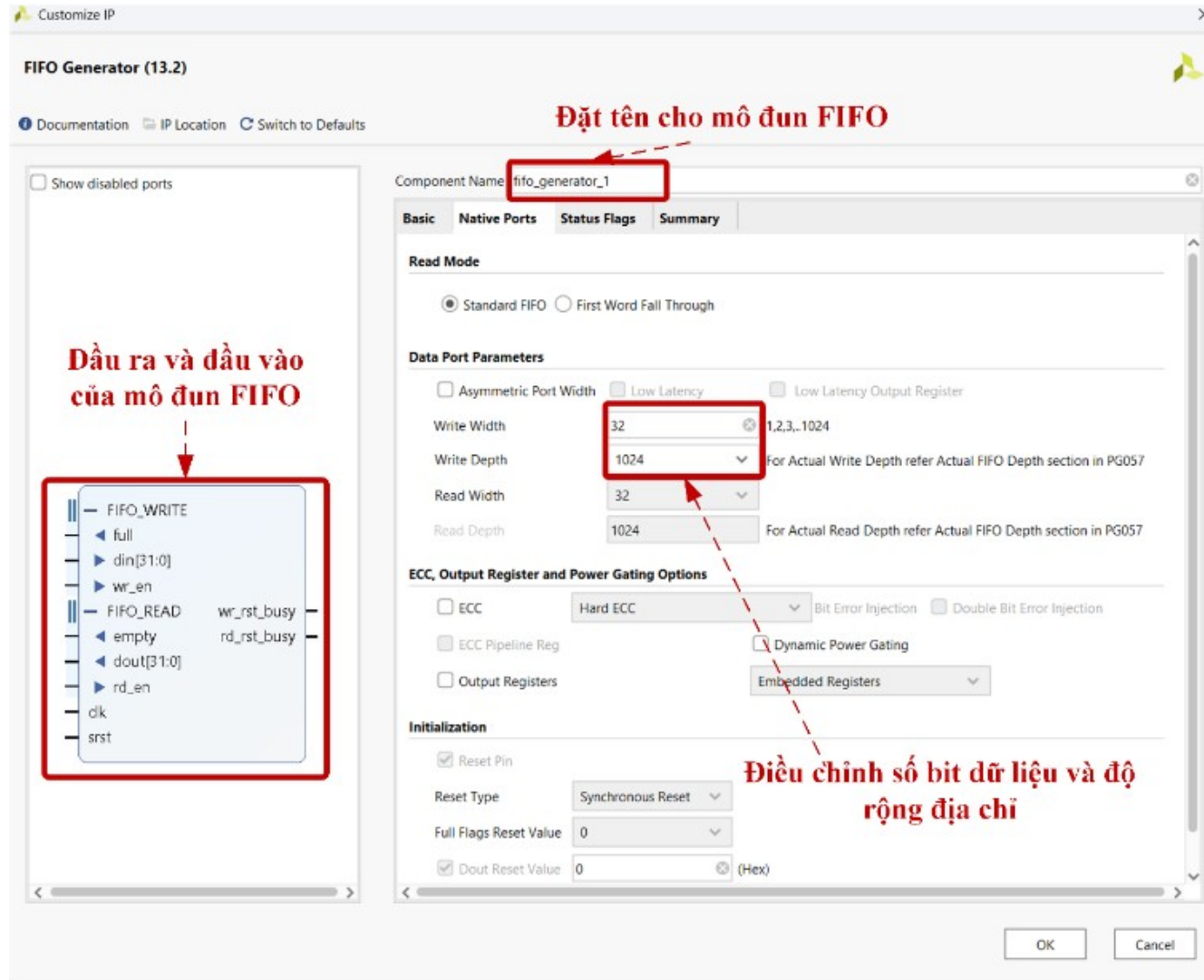
FIFO

■ Gọi mô-đun IP FIFO có sẵn:



Giao diện IP Catalog trong Vivado, nơi tìm kiếm và chọn mô-đun FIFO Generator để tạo và cấu hình FIFO trong thiết kế IP ở FPGA.

FIFO



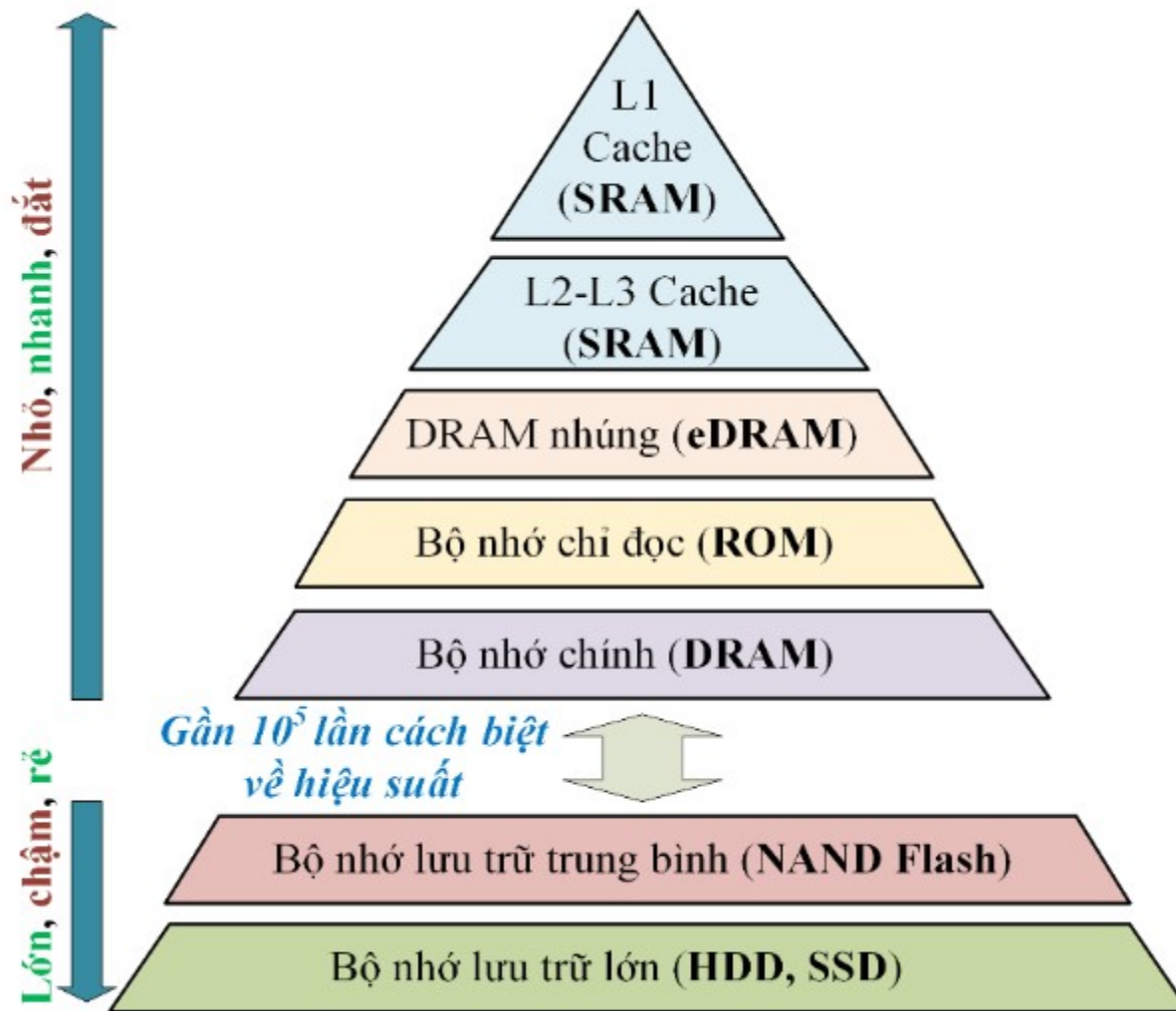
Giao diện cấu hình mô-đun FIFO Generator trong Vivado, nơi người thiết kế điều chỉnh độ rộng, độ sâu bộ nhớ FIFO và chọn các tham số liên quan đến chế độ đọc/ghi.

So sánh bộ nhớ trên Chip

■ Ưu điểm mỗi loại bộ nhớ:

- ✓ LUTRAM: Linh hoạt trong việc cấu hình và phù hợp với các ứng dụng yêu cầu lưu trữ nhỏ với độ trễ cực thấp.
- ✓ BRAM: Cung cấp dung lượng vừa phải với độ trễ thấp, thích hợp cho các bộ nhớ đệm và lưu trữ tạm thời.
- ✓ URAM: Cung cấp dung lượng lớn hơn BRAM và phù hợp với các ứng dụng yêu cầu lưu trữ dữ liệu lớn với tốc độ cao.
- ✓ FIFO: Cung cấp khả năng đồng bộ hóa và lưu trữ dữ liệu tuần tự, giúp quản lý dữ liệu giữa các phần của hệ thống có tốc độ xử lý khác nhau, và đảm bảo rằng dữ liệu đầu vào và đầu ra được xử lý đúng thứ tự.

BỘ NHỚ NGOÀI CHIP



Hệ thống phân cấp bộ nhớ ngoài chip dựa trên kích thước, tốc độ, và giá

BỘ NHỚ NGOÀI CHIP

Loại bộ nhớ	Thời gian truy cập (chu kỳ)	Dung lượng	Quản lý bởi
L1 Cache (SRAM)	3~4	64KB	Phần cứng
L2 Cache (SRAM)	10~30	256KB	Phần cứng
L3 Cache (SRAM)	30~60	2~8MB	Phần cứng
eDRAM	50~100	64~256MB	Phần cứng
ROM	100~500	1~16MB	Phần cứng
DRAM	100~300	512MB~32GB	Hệ điều hành
NAND Flash	5K~10K	8GB~32GB	Hệ điều hành
HDD	10M~20M	>1TB	Hệ điều hành

So sánh giữa các loại bộ nhớ khác nhau

BỘ NHỚ NGOÀI CHIP

■ Phân loại bộ nhớ

– eDRAM (Embedded DRAM):

- Có thời gian truy cập từ 50 đến 100 chu kỳ và dung lượng từ 64MB đến 256MB.
- Thường được sử dụng làm bộ nhớ cache tốc độ cao hoặc bộ nhớ tạm thời trong các ứng dụng cần dung lượng lớn hơn cache nhưng vẫn yêu cầu tốc độ truy cập nhanh.
- eDRAM cung cấp khả năng truy cập nhanh hơn so với DRAM tiêu chuẩn, được áp dụng trong các bộ nhớ cache lớn hoặc bộ nhớ chính trong một số hệ thống nhúng.

BỘ NHỚ NGOÀI CHIP

■ Phân loại bộ nhớ

– ROM (Read-Only Memory):

- Có thời gian truy cập từ 100 đến 500 chu kỳ và dung lượng từ 1MB đến 16MB.
- Lưu trữ dữ liệu cố định như firmware, BIOS hoặc mã khởi động, vì dữ liệu không thể thay đổi sau khi sản xuất.
- Thích hợp cho việc bảo vệ thông tin không thay đổi trong suốt vòng đời sản phẩm.

– DRAM (Dynamic RAM):

- Có thời gian truy cập từ 100 đến 300 chu kỳ và dung lượng từ 512MB đến 32GB.
- Là bộ nhớ chính của hệ thống, DRAM được quản lý bởi hệ điều hành để lưu trữ dữ liệu và chương trình đang chạy.
- Cung cấp dung lượng lớn với chi phí hợp lý, phù hợp cho các ứng dụng cần lưu trữ và truy cập dữ liệu thường xuyên.

BỘ NHỚ NGOÀI CHIP

■ Phân loại bộ nhớ

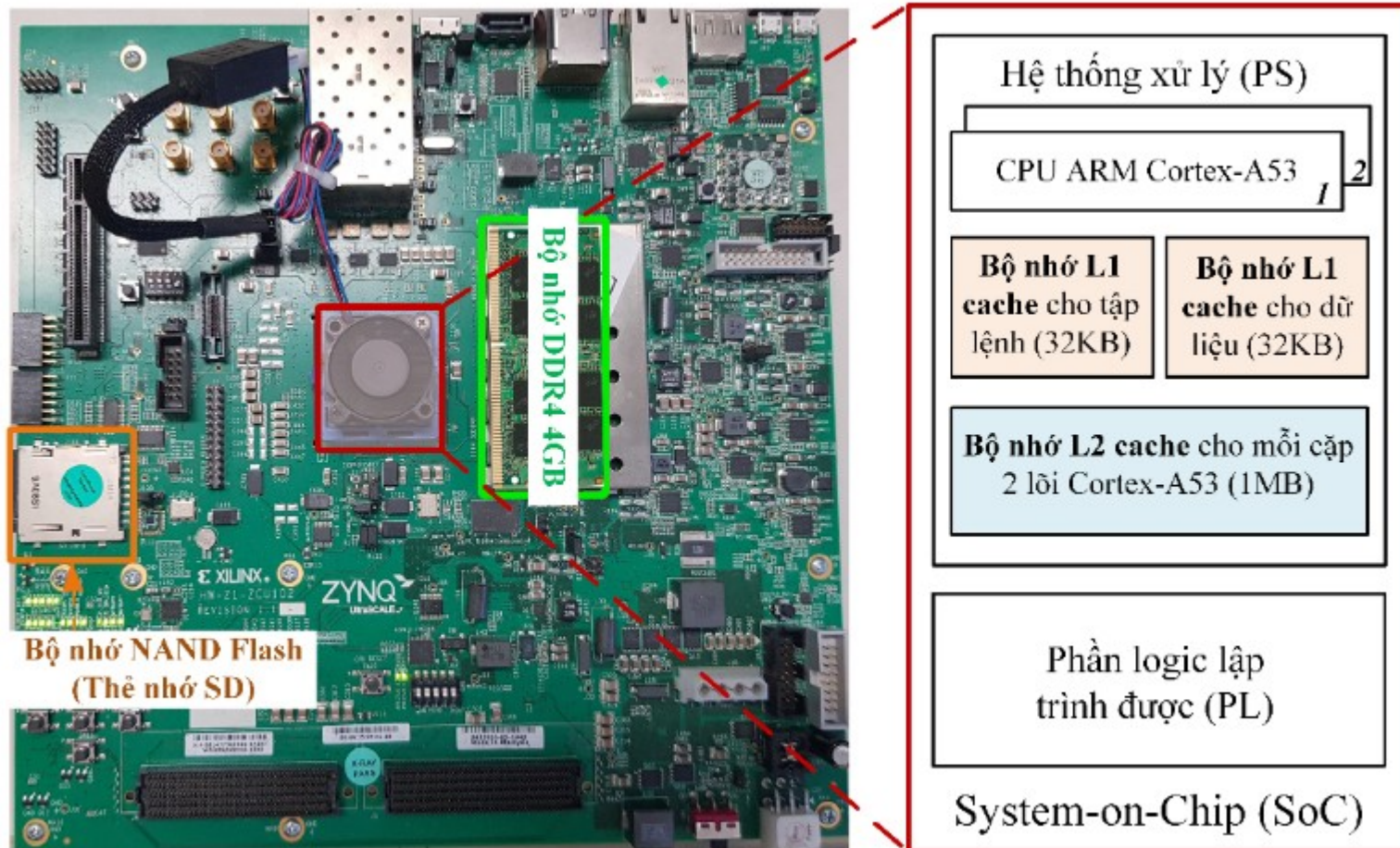
– NAND Flash:

- Có thời gian truy cập từ 5.000 đến 10.000 chu kỳ và dung lượng từ 8GB đến 32GB.
- Là bộ nhớ không bay hơi, nó thường được sử dụng trong các thiết bị lưu trữ như USB, SSD và thiết bị nhúng.
- Chi phí thấp hơn DRAM, phù hợp cho lưu trữ dữ liệu dài hạn và dữ liệu di động, mặc dù tốc độ truy cập chậm hơn.

– HDD (Hard Disk Drive):

- HDD có thời gian truy cập chậm nhất, từ 10 triệu đến 20 triệu chu kỳ, nhưng dung lượng rất lớn, thường lớn hơn 1TB (Terabyte).
- Lưu trữ dữ liệu dài hạn với chi phí thấp, phù hợp cho các ứng dụng lưu trữ dữ liệu lớn như cơ sở dữ liệu, lưu trữ dữ liệu sao lưu, và các thư viện số.

BỘ NHỚ NGOÀI CHIP



Các bộ nhớ ngoài chip trên FPGA ZCU102, bao gồm bộ nhớ DDR4 4GB, bộ nhớ NAND Flash, và các bộ nhớ cache L1, L2 của CPU ARM Cortex-A53 trong hệ thống SoC.

Bộ nhớ đệm (Cache) L1, L2, L3

■ Cơ chế lưu trữ:

- Cache lưu trữ dữ liệu dưới dạng "cache lines", mỗi line có độ dài cố định tùy thuộc vào phần cứng.
 - Ví dụ, một cache line có độ dài 32 byte và được căn chỉnh trên biên độ 32 byte.
- Khi CPU truy cập một địa chỉ bộ nhớ, toàn bộ cache line chứa địa chỉ đó sẽ được kéo vào cache, tăng tốc độ truy cập dữ liệu.
- Khi dữ liệu được tìm thấy trong cache, gọi là cache hit, dữ liệu có thể truy cập ngay lập tức, tiết kiệm thời gian. Ngược lại, nếu dữ liệu không có trong cache, gọi là cache miss, CPU sẽ truy xuất dữ liệu từ bộ nhớ chính và tải nó vào cache.
- Trong quá trình này, cache sẽ nạp một vùng bộ nhớ liên kết, chứa một hoặc nhiều từ vật lý. Từ vật lý là đơn vị cơ bản để chuyển dữ liệu giữa bộ xử lý và bộ nhớ cache.

Bộ nhớ đệm (Cache) L1, L2, L3

■ Giảm lỗi cache:

- Để giảm tỷ lệ lỗi của cache và tăng hiệu suất, nguyên lý tính cục bộ (locality) cho rằng các chương trình thường tái sử dụng dữ liệu và lệnh gần đây. Nguyên lý này chia thành hai loại:
 - Tính cục bộ theo thời gian (temporal locality): chỉ khả năng các dữ liệu vừa truy xuất sẽ được truy cập lại trong tương lai gần.
 - Tính cục bộ theo không gian (spatial locality): ám chỉ các mục có địa chỉ gần nhau thường xuyên được truy xuất cùng lúc.

■ Ví dụ về tính cục bộ theo không gian:

- Việc tham chiếu các phần tử mảng liên tiếp, như trong mô hình tham chiếu stride-1. Tính cục bộ theo thời gian thể hiện qua việc tham chiếu một biến, ví dụ như sum, trong mỗi vòng lặp. Đoạn mã lặp qua các phần tử mảng thể hiện cả hai loại tính cục bộ: truy cập mảng cho tính cục bộ theo không gian và tham chiếu biến sum cho tính cục bộ theo thời gian. Tương tự, các lệnh thực thi liên tiếp cũng đại diện cho tính cục bộ theo không gian, trong khi việc lặp lại vòng lặp minh họa tính cục bộ theo thời gian.

Bộ nhớ đệm (Cache) L1, L2, L3

Giả sử bộ nhớ cache có kích thước 4 byte và mỗi block cache chứa 7 từ (words)

```
int sumarrayrows (int a[M][N])
{
    int i, j, sum = 0;

    for ( i = 0; i < M; i++)
        for ( j = 0; j < N; j++)
            sum += a[ i ][ j ];

    return sum;
}
```

Tỷ lệ lỗi cache là = $1/8 = 12.5\%$
(a)

```
int sumarrayrows (int a[M][N])
{
    int i, j, sum = 0;

    for ( j = 0; j < N; j++)
        for ( i = 0; i < M; i++)
            sum += a[ i ][ j ];

    return sum;
}
```

Tỷ lệ lỗi cache là = 100%
(b)

Tỷ lệ lỗi cache khi truy cập mảng theo thứ tự khác nhau với bộ nhớ cache 4 byte và mỗi block chứa 7 từ. (a) Lỗi cache 12.5%, (b) Lỗi cache 100%.

Bộ nhớ đệm (Cache) L1, L2, L3

Giải thích ví dụ:

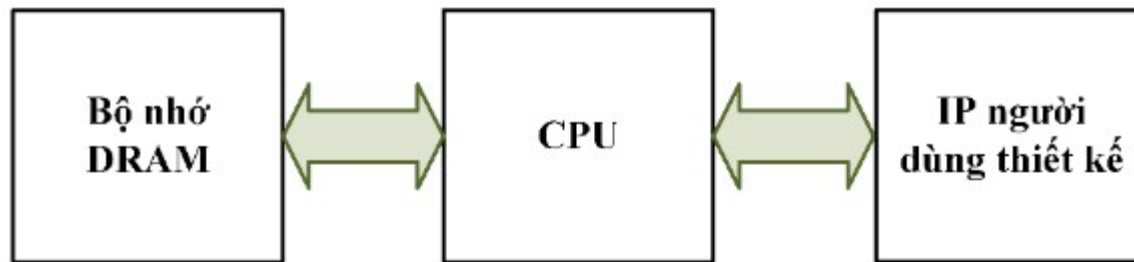
■ Hình 3.28 minh họa tỷ lệ lỗi cache khi truy cập mảng với bộ nhớ cache 4 byte, mỗi block chứa 7 từ. Trong hàm `sumarrayrows` (bên trái), dữ liệu được truy cập theo chiều dọc, tận dụng tính cục bộ theo không gian và dẫn đến tỷ lệ lỗi cache thấp (12.5%). Tỷ lệ này được tính từ việc mỗi block chứa 7 từ, nên khi truy cập phần tử ngoài 7 từ đầu tiên, sẽ xảy ra lỗi cache. Tổng số lỗi là 1 trong 8 lần truy cập (1/8).

■ Ngược lại, trong hàm `sumarraycols` (bên phải), việc truy cập theo cột tạo ra các địa chỉ không liên tiếp, dẫn đến tỷ lệ lỗi cache cao (100%). Do đó, duyệt qua các hàng (như trong `sumarrayrows`) là cách tối ưu hơn, giúp cải thiện việc sử dụng bộ nhớ cache và giảm độ trễ do lỗi cache.

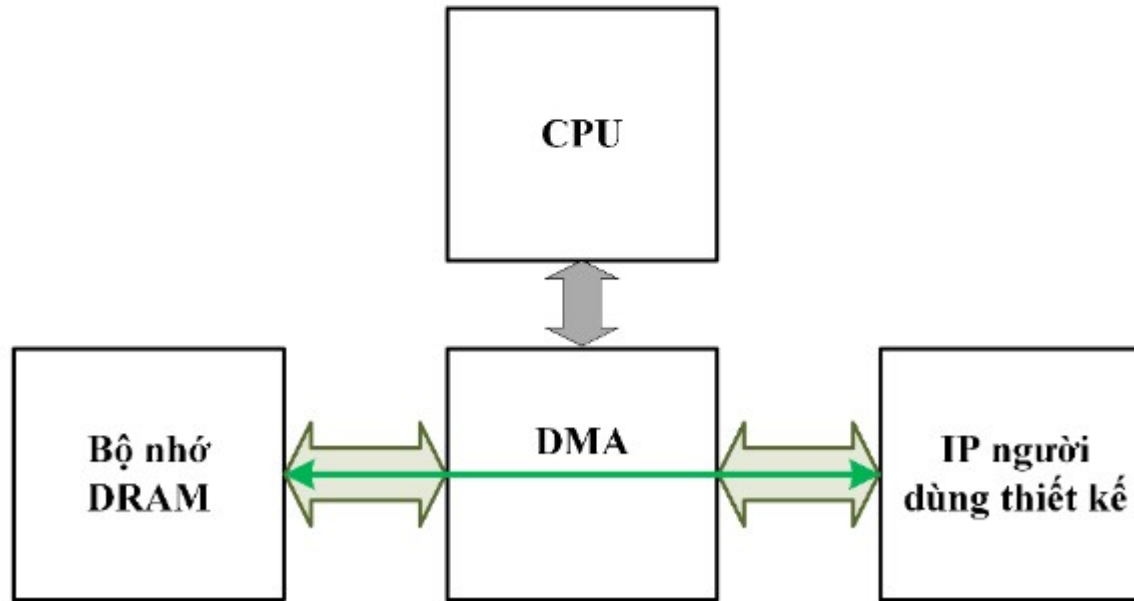
Bộ nhớ DRAM

- Bộ nhớ truy cập ngẫu nhiên động (DRAM) là loại bộ nhớ có khả năng lưu trữ lớn, thường từ vài GB đến hàng chục GB. Mặc dù tốc độ truy xuất chậm hơn các bộ nhớ cache như L1, L2, L3, DRAM vẫn đóng vai trò quan trọng trong hệ thống SoC FPGA.
- Tốc độ truy xuất: DRAM chậm hơn cache, nhưng cache sẽ truy cập DRAM khi không có dữ liệu cần thiết, giúp tối ưu hóa tốc độ xử lý.
- Chức năng: DRAM lưu trữ dữ liệu tạm thời và các chương trình lớn, cung cấp không gian bộ nhớ linh hoạt cho các ứng dụng phức tạp.
- Kết nối và ứng dụng: Trong hệ thống SoC FPGA, DRAM thường được kết nối với các IP thiết kế để cung cấp dung lượng lưu trữ cao và hỗ trợ truy xuất nhanh cho các phép toán tính toán hoặc xử lý tín hiệu.
- Ví dụ ứng dụng: Trong AI, DRAM lưu trữ các tham số huấn luyện như trọng số và độ lệch, giúp tiết kiệm chi phí. Trong xử lý ảnh, DRAM lưu trữ tạm thời số lượng lớn ảnh đầu vào và đầu ra, truyền liên tiếp vào IP để tính toán.

Bộ nhớ DRAM



(a)



(b)

Các phương thức truyền dữ liệu giữa bộ nhớ DRAM và IP người dùng thiết kế (a) PIO, (b) DMA.

Bộ nhớ DRAM

Cách sử dụng DRAM thực hiện qua hai phương thức chính: PIO (Programmed I/O) và DMA (Direct Memory Access).

■PIO (Programmed I/O): CPU trực tiếp điều khiển truyền tải dữ liệu giữa DRAM và các IP thông qua các lệnh I/O.

- Ưu điểm: Dễ triển khai.
- Nhược điểm: Gây tải nặng cho CPU vì nó phải xử lý tất cả các thao tác truyền nhận dữ liệu.

■DMA (Direct Memory Access): Cho phép truyền tải dữ liệu giữa DRAM và các IP mà không cần sự can thiệp của CPU.

- Ưu điểm: Giảm tải cho CPU, tối ưu hóa băng thông, và truyền tải nhanh chóng và hiệu quả mà không cần chờ đợi CPU.

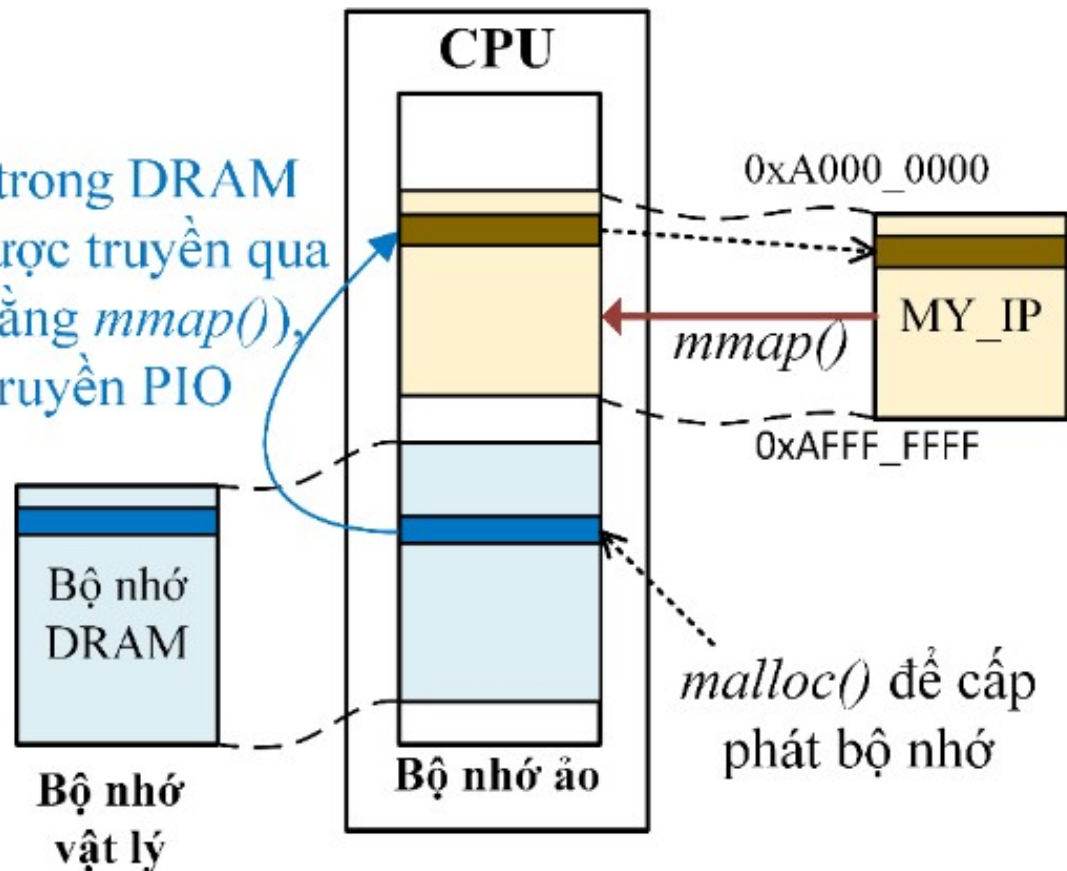
■PIO thích hợp cho các tín hiệu điều khiển cơ bản của IP. DMA được ưu tiên khi cần truyền nhận dữ liệu lớn giữa DRAM và các bộ nhớ trên chip (như URAM, BRAM, FIFO), giúp giảm độ trễ và tối ưu hóa hiệu suất hệ thống.



(b) Địa chỉ được chỉ định của MY_IP trên SoC.

Bộ nhớ DRAM

Dữ liệu được khởi tạo trong DRAM
hông qua *malloc()* sẽ được truyền qua
MY_IP (được ánh xạ bằng *mmap()*),
và đây là quá trình truyền PIO



Quá trình truyền PIO từ DRAM vào MY_IP thông qua ánh xạ bộ nhớ *mmap()*.

Bộ nhớ DRAM

```
#define MY_IP_BASE_ADDRESS 0xA0000000 // Địa chỉ của MY_IP trong bộ nhớ
#define MY_IP_SIZE 0x1000 // Kích thước vùng bộ nhớ của MY_IP
#define MY_IP_REGISTER_OFFSET 0x04 // Địa chỉ offset cho register của MY_IP
#define DRAM_SIZE 0x1000 // Kích thước vùng bộ nhớ DRAM
```

Định nghĩa vùng địa chỉ và kích thước địa chỉ của MY_IP

```
int main() {
    // Cấp phát bộ nhớ cho dữ liệu trong DRAM bằng malloc()
    uint32_t* dram_data = (uint32_t*) malloc(DRAM_SIZE);
    if (!dram_data) { perror("malloc() failed"); exit(1); } // Kiểm tra xem malloc có thành công không

    // Khởi tạo dữ liệu trong DRAM
    for (int i = 0; i < DRAM_SIZE / sizeof(uint32_t); i++) {
        dram_data[i] = 0x1000 + i; } // Dữ liệu mẫu
```

Cấp phát và khởi tạo dữ liệu trong bộ nhớ ảo, và dữ liệu này có thể được ánh xạ vào bộ nhớ vật lý (như DRAM)

```
// Mở /dev/mem và ánh xạ MY_IP vào bộ nhớ người dùng bằng mmap()
int mem_fd = open("/dev/mem", O_RDWR | O_SYNC);
if (mem_fd == -1) {
    perror("Failed to open /dev/mem"); exit(1); } // Kiểm tra việc mở /dev/mem có thành công không
```

```
// Ánh xạ MY_IP vào bộ nhớ ảo
volatile uint32_t* my_ip_base = (volatile uint32_t*) mmap(NULL, MY_IP_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, mem_fd, MY_IP_BASE_ADDRESS);
close(mem_fd); // Đóng tệp /dev/mem
```

Mở /dev/mem và ánh xạ MY_IP vào bộ nhớ người dùng bằng mmap()

```
// Truyền dữ liệu từ DRAM vào MY_IP
for (int i = 0; i < DRAM_SIZE / sizeof(uint32_t); i++) {
    my_ip_base[i] = dram_data[i]; // Truyền dữ liệu từ DRAM vào MY_IP
}
```

```
// Bước 5: Đọc lại dữ liệu từ MY_IP (để kiểm tra)
for (int i = 0; i < DRAM_SIZE / sizeof(uint32_t); i++) {
    dram_data[i] = my_ip_base[i]; // Lấy dữ liệu từ MY_IP và lưu vào DRAM
}
```

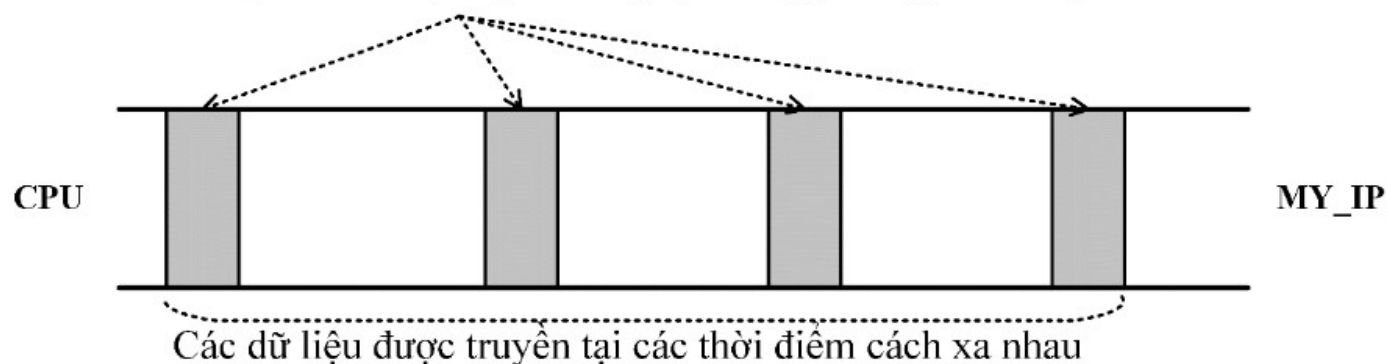
Ghi và đọc dữ liệu cho MY_IP từ vùng nhớ ảo được ánh xạ vào bộ nhớ vật lý (DRAM)

```
free(dram_data); // Giải phóng bộ nhớ đã cấp phát cho DRAM
munmap((void*)my_ip_base, MY_IP_SIZE); // Giải phóng bộ nhớ đã ánh xạ cho MY_IP
return 0;
}
```

Code C mô tả quá trình truyền PIO từ DRAM vào MY_IP thông qua ánh xạ bộ nhớ mmap().

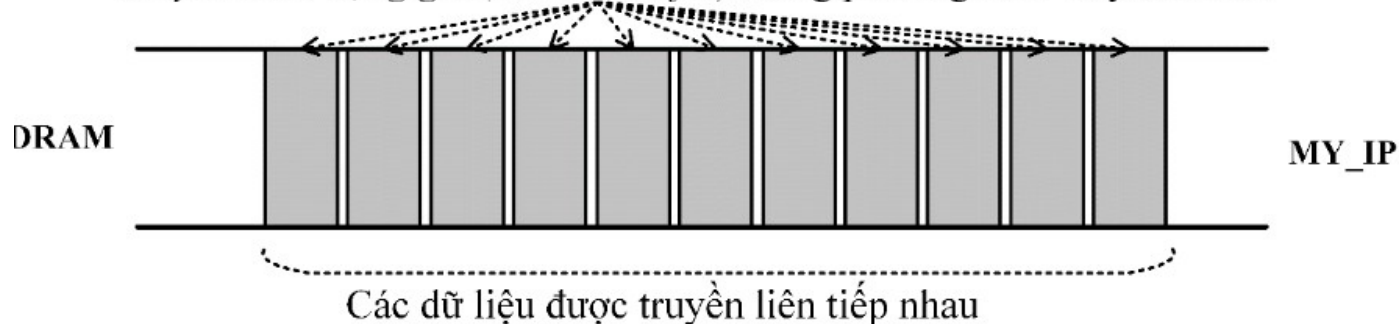
Bộ nhớ DRAM - DMA

Truyền đơn lẻ (single transfer) trong phương thức truyền PIO



(a)

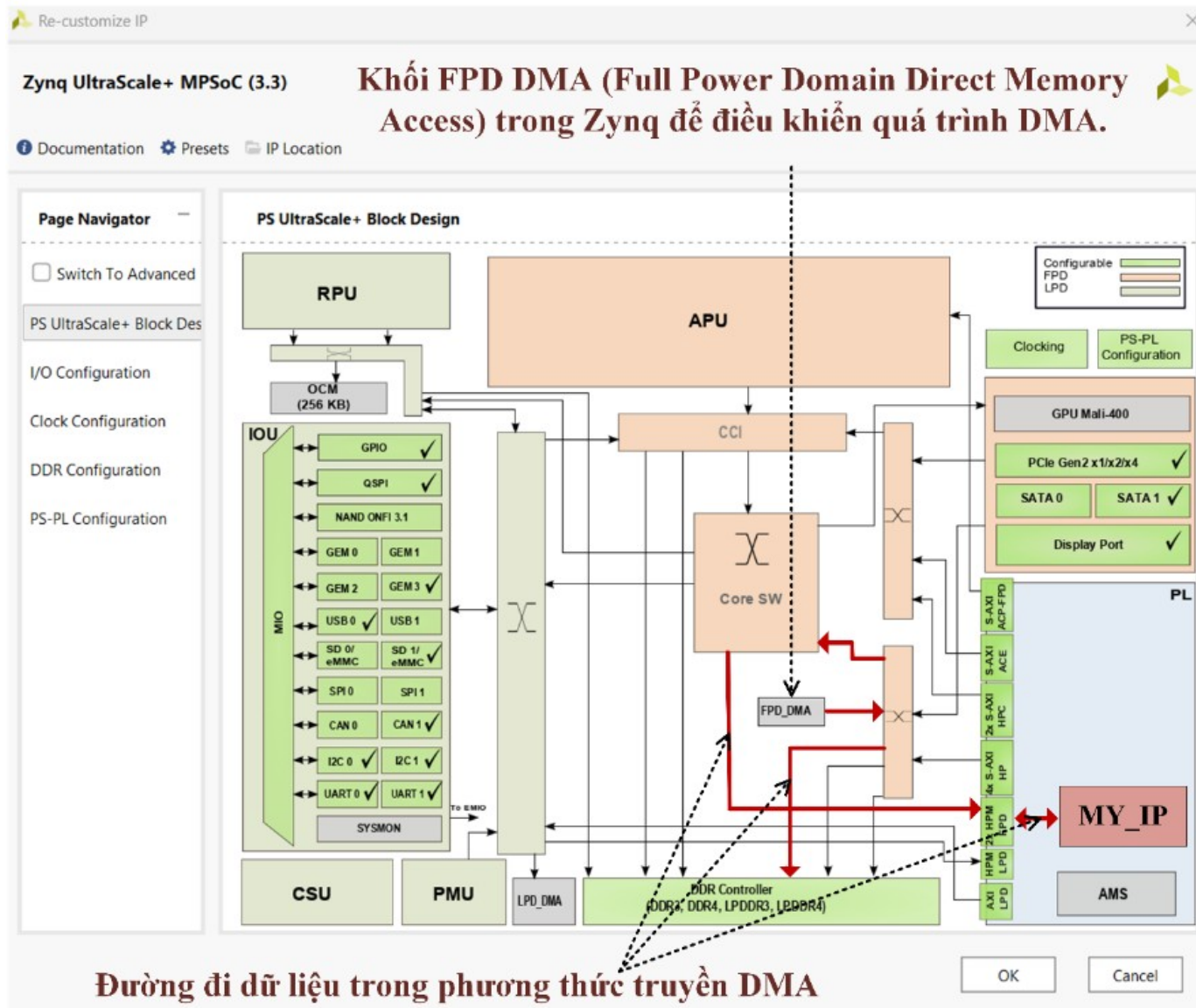
Truyền theo dạng gói (burst transfer) trong phương thức truyền DMA



(b)

(a) Truyền đơn lẻ (single transfer) trong phương thức truyền PIO và
(b) truyền theo dạng gói (burst transfer) trong phương thức truyền DMA

Bộ nhớ DRAM - DMA



Khối FPD DMA trong hệ thống xử lý (ZYNQ PS) hỗ trợ truyền DMA từ DRAM sang MY_IP.

Bộ nhớ DRAM - DMA

```
#define MY_IP_BASE_ADDRESS 0xA0000000 // Địa chỉ của MY_IP
#define MY_IP_SIZE 0x1000 // Kích thước vùng bộ nhớ của MY_IP
#define MY_IP_REGISTER_OFFSET 0x04 // Địa chỉ offset cho register của MY_IP
#define DRAM_SIZE 0x1000 // Kích thước vùng bộ nhớ DRAM
#define DMA_BASE_PHYS 0x00000000FD500000LL // Địa chỉ DMA
#define DMA_MMAP_SIZE 0x0000000000010000LL // Kích thước bộ nhớ ánh xạ DMA
```

Định nghĩa vùng địa chỉ và kích thước địa chỉ của MY_IP

```
int main() {
    // Ánh xạ DRAM vào bộ nhớ ảo
    int fd_ddr = open("/dev/mem", O_RDWR | O_SYNC);
    if (fd_ddr == -1) { perror("Failed to open /dev/mem"); exit(1); }
    uint32_t* dram_data = (uint32_t*) mmap(NULL, DRAM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd_ddr, DRAM_BASE_PHYS);
    if (dram_data == MAP_FAILED) { perror("mmap() failed"); close(fd_ddr); exit(1); }
    close(fd_ddr);
```

```
// Khởi tạo dữ liệu trong DRAM
for (int i = 0; i < DRAM_SIZE / sizeof(uint32_t); i++) {
    dram_data[i] = 0x1000 + i; // Dữ liệu mẫu
}
```

Cấp phát và khởi tạo dữ liệu trong bộ nhớ ảo thông qua mmap(), và dữ liệu này được ánh xạ vào bộ nhớ vật lý DRAM

```
// Mở /dev/mem và ánh xạ DMA vào bộ nhớ ảo
int mem_fd = open("/dev/mem", O_RDWR | O_SYNC);
if (mem_fd == -1) { perror("Failed to open /dev/mem"); exit(1); }
// Ánh xạ DMA vào bộ nhớ ảo
volatile uint32_t* dma_base = (volatile uint32_t*) mmap(NULL, DMA_MMAP_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, mem_fd, DMA_BASE_PHYS);
close(mem_fd); // Đóng tệp /dev/mem
```

Mở /dev/mem và ánh xạ DMA vào bộ nhớ người dùng bằng mmap()

```
// Truyền dữ liệu từ DRAM vào MY_IP
dma_base->ZDMA_CH_SRC_DSCR_WORD0 = (uint64_t) dram_data; // Địa chỉ nguồn (DRAM)
dma_base->ZDMA_CH_SRC_DSCR_WORD2 = DRAM_SIZE; // Kích thước dữ liệu nguồn
dma_base->ZDMA_CH_DST_DSCR_WORD0 = MY_IP_BASE_ADDRESS + MY_IP_REGISTER_OFFSET; // Địa chỉ đích (MY_IP)
dma_base->ZDMA_CH_DST_DSCR_WORD2 = DRAM_SIZE; // Kích thước dữ liệu đích
dma_base->ZDMA_CH_CTRL2 = 1; // Bật DMA để truyền dữ liệu
while (dma_base->ZDMA_CH_STATUS & 3); // Chờ cho quá trình DMA truyền dữ liệu từ DRAM vào MY_IP hoàn tất
// Truyền dữ liệu từ MY_IP vào DRAM
dma_base->ZDMA_CH_SRC_DSCR_WORD0 = MY_IP_BASE_ADDRESS + MY_IP_REGISTER_OFFSET; // Địa chỉ nguồn (MY_IP)
dma_base->ZDMA_CH_SRC_DSCR_WORD2 = DRAM_SIZE; // Kích thước dữ liệu nguồn
dma_base->ZDMA_CH_DST_DSCR_WORD0 = (uint64_t) dram_data; // Địa chỉ đích (DRAM)
dma_base->ZDMA_CH_DST_DSCR_WORD2 = DRAM_SIZE; // Kích thước dữ liệu đích
dma_base->ZDMA_CH_CTRL2 = 1; // Bật DMA để truyền dữ liệu
while (dma_base->ZDMA_CH_STATUS & 3); // Chờ cho quá trình DMA truyền dữ liệu từ MY_IP về DRAM hoàn tất
```

Truyền DMA từ DRAM vào MY_IP hoặc đọc dữ liệu từ MY_IP vào DRAM

```
return 0;
```

Code C mô tả quá trình truyền DMA từ DRAM vào MY_IP thông qua FPD DMA.

Bộ nhớ NAND flash

Đặc điểm:

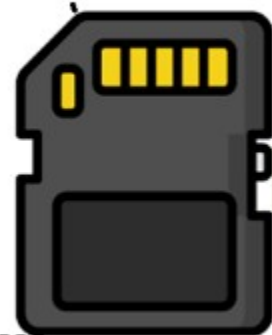
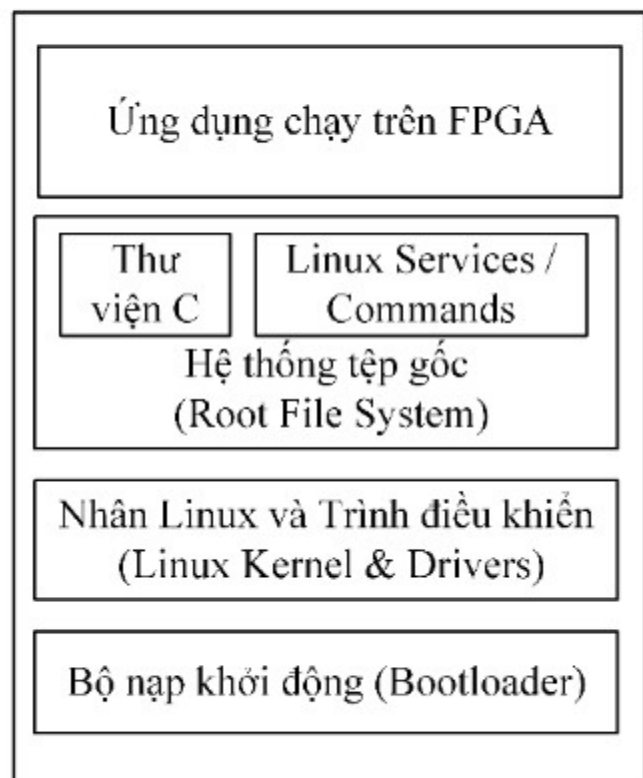
- NAND Flash được triển khai trong các thẻ nhớ như SD, Mini SD và Micro SD, với khả năng thay thế ổ đĩa, mang lại lợi ích về mật độ lưu trữ cao và tốc độ truyền tải nhanh.
- Thẻ nhớ SD với NAND Flash cung cấp mật độ bit lớn hơn so với các công nghệ Flash khác như NOR, giúp lưu trữ nhiều dữ liệu hơn trong không gian vật lý nhỏ gọn.
- Tốc độ đọc/ghi khoảng từ 22 MBps (đọc) đến 18 MBps (ghi), phù hợp cho các ứng dụng như lưu trữ dữ liệu trong FPGA.

Bộ nhớ NAND flash

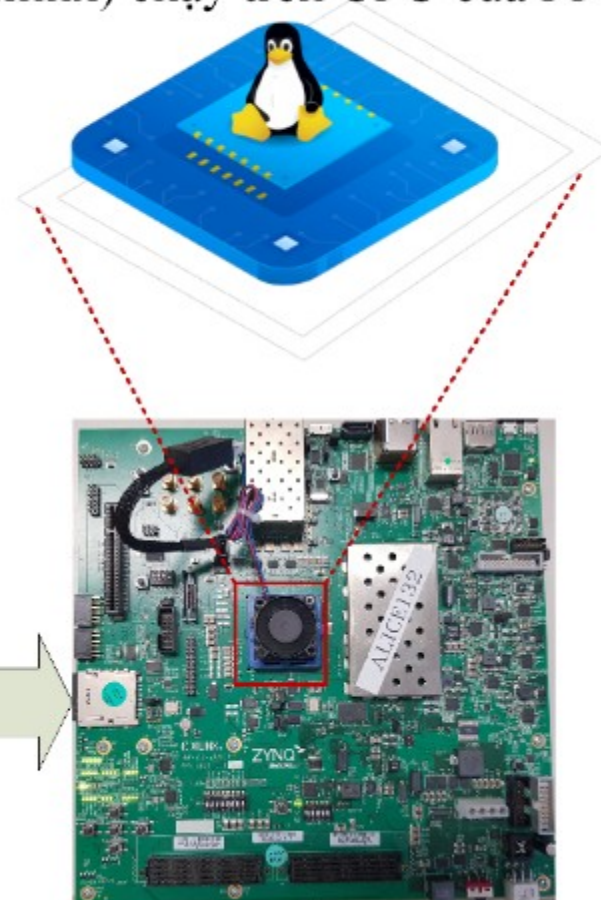
Cách hoạt động:

Hệ điều hành nhúng (Ví dụ Petalinux ở các Zynq FPGA dòng Xilinx) chạy trên CPU của FPGA

Dữ liệu cho hệ điều hành nhúng



Thẻ nhớ SD



NAND Flash lưu trữ dữ liệu cho hệ điều hành nhúng trên SoC FPGA như một ổ cứng trên các máy tính thông thường.

Bộ nhớ NAND flash



Ứng dụng của thẻ nhớ SD trong việc lưu trữ dữ liệu cho IP thiết kế trong SoC và quy trình truyền dữ liệu từ thẻ nhớ SD lên DRAM và MY_IP.

Thẻ nhớ SD trong hệ thống SoC cho phép lưu trữ dữ liệu quan trọng cho các IP thiết kế. Dữ liệu được lưu trong các tệp và đọc vào bộ nhớ DRAM qua lệnh truy xuất tiêu chuẩn như `open("file")`. Sau khi có trong DRAM, dữ liệu được truyền từ DRAM đến MY_IP qua DMA, giảm tải cho CPU và tối ưu hóa băng thông. Quá trình này không cần can thiệp của CPU, đảm bảo hiệu suất truyền tải nhanh chóng và hiệu quả giữa bộ nhớ và các IP xử lý.

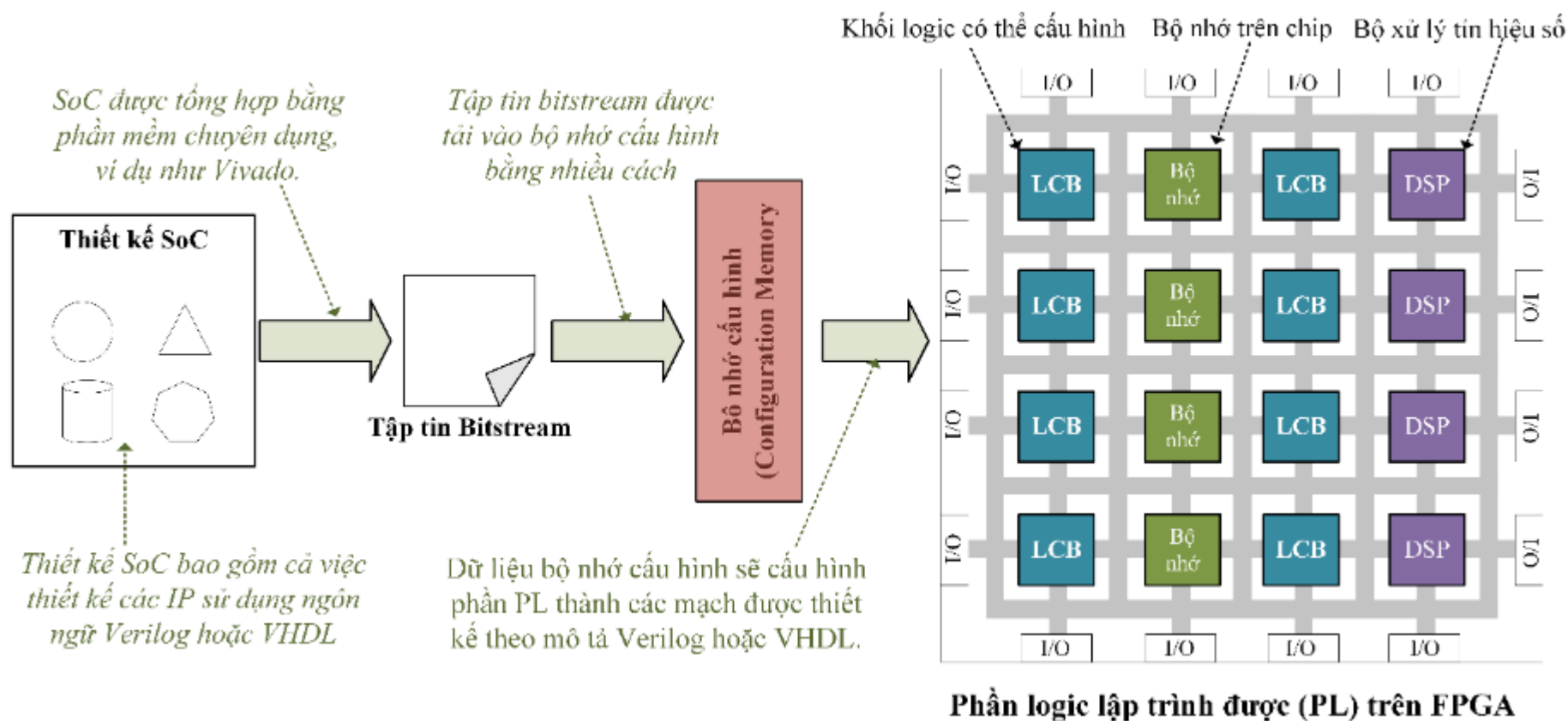
Bộ nhớ cấu hình (Configuration Memory)

Đặc điểm:

■ Bộ nhớ cấu hình (Configuration Memory) trong FPGA là phần quan trọng để định hình và cấu hình lại phần logic có thể lập trình (PL).

■ Khi thiết kế SoC, các mô-đun IP được mô tả bằng Verilog hoặc VHDL, tạo ra tập tin bitstream chứa thông tin cấu hình cần thiết. Bộ nhớ cấu hình lưu trữ thông tin này, và khi tải vào FPGA, nó định hình lại các khối logic trong chip, giúp FPGA thực hiện các nhiệm vụ theo yêu cầu hệ thống.

Bộ nhớ cấu hình (Configuration Memory)



Quy trình cấu hình SoC dùng bộ nhớ cấu hình trên FPGA.

Câu hỏi và Bài tập

1. Tại sao bộ nhớ BRAM thường được sử dụng cho các ứng dụng yêu cầu độ trễ thấp và tốc độ xử lý cao? Giả sử bạn cần lưu trữ 8MB dữ liệu trong FPGA, mỗi khối BRAM có dung lượng 18Kb. Tính số lượng BRAM cần thiết để lưu trữ 8MB dữ liệu.
2. Giải thích cách thức sử dụng bộ nhớ DRAM trong hệ thống SoC FPGA để hỗ trợ các ứng dụng tính toán và xử lý tín hiệu? Trong hệ thống SoC có tốc độ xung nhịp 330 MHz và truyền DMA dữ liệu từ DRAM với tốc độ 40 Gbps. Độ rộng dữ liệu mỗi chu kỳ là 128 bit. Nếu bạn cần truyền 256MB dữ liệu từ DRAM, tính thời gian truyền nhận dữ liệu từ DRAM tới các IP.